
Resolución de los cuatro dolores estructurales mediante patrones de diseño

Diagnóstico sobre el código actual, fundamento teórico y solución implementada para **State**, **Decorator/Proxy**, **Command** y **Template Method**.

SISTEMA

PCFORGE — Sistema de información de PC Factory

ARQUITECTURA

N capas · BE · DAL · MPP · BLL · SER · UI

PROCESOS DE NEGOCIO

RFN1 Venta de computadoras · RFN2 Compra de insumos

REPOSITORIO

IngSoftValdezAlegre

PLATAFORMA

C# · .NET Framework 4.8.1 · WinForms · SQL Server 2022

AUTOR

Thiago Valdez · UAI

Contenido

1. Marco teórico común

Por qué un patrón y no un *if* más
Los tres principios que atacan los cuatro dolores
Criterio de aplicación y riesgo de sobre-ingeniería

2. Cuadro de situación: los cuatro dolores

Evidencia cuantificada sobre el código actual

3. Dolor I — Transiciones de estado dispersas → State

Diagnóstico · Fundamento GoF · Diseño · Código · Migración

4. Dolor II — Seguridad y auditoría por convención → Decorator / Proxy

Diagnóstico · Fundamento GoF · Diseño · Código · Variante con proxy dinámico

5. Dolor III — Compensación manual duplicada → Command

Diagnóstico · Fundamento GoF · Diseño · Código · Relación con transacciones

6. Dolor IV — Duplicación en la capa DAL → Template Method

Diagnóstico · Fundamento GoF · Diseño · Código · Extensiones habilitadas

7. Plan de implementación

Orden, dependencias entre refactors y estrategia de verificación

8. Bibliografía

Marco teórico común

Qué hace que estos cuatro sean patrones y no simplemente refactorizaciones.

1.1 Por qué un patrón y no un *if* más

Un patrón de diseño no es una técnica de programación: es una **solución nombrada a un problema recurrente en un contexto determinado**, con consecuencias conocidas. Gamma, Helm, Johnson y Vlissides lo definen como la tríada *problema — solución — consecuencias*; el aporte del catálogo no es el código sino el **vocabulario compartido** y la previsibilidad de sus efectos.

Eso importa acá por una razón práctica. Los cuatro problemas que trata este documento **ya tienen solución en el código**: todos funcionan. El sistema vende, produce, compra y recibe. Lo que no tienen es una solución *que resista el cambio*: cada una está escrita de forma tal que agregar un caso obliga a modificar código que ya funciona, en varios lugares a la vez, sin que nada garantice que se modificaron todos.

EL CRITERIO OPERATIVO

Un patrón se justifica cuando existe un **eje de variación identificado** —algo que ya cambió o que se sabe que va a cambiar— y el diseño actual obliga a tocar N lugares para acompañarlo. Si no hay eje de variación, el patrón agrega indirección sin beneficio: eso es sobre-ingeniería, y en un trabajo académico se nota más que la ausencia del patrón.

1.2 Los tres principios que atacan los cuatro dolores

ABIERTO/CERRADO (OCP) — MEYER, 1988; MARTIN, 1996

Una entidad de software debe estar **abierta a la extensión y cerrada a la modificación**. La prueba concreta es: *¿puedo agregar un caso nuevo escribiendo código nuevo, sin editar código existente?*

Hoy, en PCFORGE, la respuesta es no en los cuatro dolores: agregar un estado a la orden de producción obliga a editar tres archivos; agregar un servicio de negocio obliga a repetir a mano el control de patente y la auditoría; agregar un paso reversible obliga a extender un `try/catch` existente; agregar un DAL obliga a copiar 30 líneas. **Los cuatro patrones convierten esos "editar" en "agregar"**.

RESPONSABILIDAD ÚNICA (SRP)

Una clase debe tener **una sola razón para cambiar**. `OrdenProduccionBLL06AV` tiene hoy al menos cuatro: cambia si cambian las reglas de transición, si cambia el esquema de persistencia, si cambia la política de auditoría y si cambia la estrategia de compensación ante fallos. State, Decorator y Command reparten exactamente esas cuatro razones en cuatro lugares distintos.

GRASP: POLIMORFISMO E INDIRECCIÓN — LARMAN

Larman enuncia el patrón *Polimorfismo* justamente como respuesta a la pregunta "¿cómo manejar alternativas según el tipo?": la respuesta es **asignar la responsabilidad a los tipos mediante operaciones polimórficas, no mediante lógica condicional**. Un `switch` sobre un enum de estados es el antipatrón que

ese principio nombra. También aplica *Variaciones Protegidas*: envolver el punto de inestabilidad en una interfaz estable —que es literalmente lo que hacen Decorator y Template Method acá.

1.3 Riesgo de sobre-ingeniería: por qué estos cuatro y no doce

Los cuatro se seleccionaron con el mismo criterio de §1.1, y cada uno tiene evidencia numérica en el código que se detalla en el capítulo 2. Los patrones que **no** se proponen acá (Visitor, Flyweight, Interpreter, Abstract Factory de persistencia) fueron descartados porque el eje de variación que los justificaría no existe en este dominio: PC Factory no cambia de motor de base de datos, no tiene una jerarquía de entidades sobre la que agregar operaciones, no maneja volúmenes que justifiquen compartir instancias y no interpreta ningún lenguaje.

Cuadro de situación

Los cuatro dolores, con su evidencia medida sobre el código actual.

#	Dolor estructural	Evidencia en el repositorio	Patrón	Principio violado
I	La máquina de estados está escrita tres veces y ninguna es la fuente de verdad.	4 guardas <code>if (x.Estado != ...)</code> + 1 <code>switch</code> de 6 ramas en <code>OrdenProduccionBLL06AV</code> ; cascada equivalente en <code>ComprasControl.EstadoDe()</code> ; repetido para 4 enums de estado (<code>Venta</code> , <code>OrdenProduccion</code> , <code>OrdenCompra</code> , <code>Cotizacion</code>).	STATE	OCP · SRP · GRASP · Polimorfismo
II	Autorización y auditoría dependen de que el programador se acuerde de escribirlas.	<code>ExigirPatente</code> existe en 1 de 6 clases BLL; <code>AuditoriaPcFactory06AV</code> se invoca a mano en ~25 métodos y es el único disparador de <code>IntegridadBLL06AV.RecalcularSeguro()</code> .	DECORATOR PROXY	SRP · DIP · GRASP · Variaciones Protegidas
III	La consistencia se sostiene con compensación manual copiada, y un paso queda fuera.	3 bloques <code>try/catch</code> con lista acumuladora en <code>VentasBLL06AV</code> y <code>OrdenProduccionBLL06AV</code> ; 0 usos de <code>SqlTransaction</code> en todo el repositorio; <code>LiberarLinea()</code> fuera del alcance de la compensación.	COMMAND	DRY · SRP · OCP
IV	El esqueleto del acceso a datos está duplicado en toda la capa.	<code>EjecutarSP</code> + <code>EjecutarSPNonQuery</code> idénticos en 11 de 15 clases de DAL/ (~330 líneas duplicadas), todas sin <code>using</code> ni <code>finally</code> .	TEMPLATE METHOD	DRY · OCP

DEPENDENCIA ENTRE LOS CUATRO

No son independientes y conviene atacarlos en orden. **El IV habilita al III**: centralizar el esqueleto del DAL es lo que después permite introducir `SqlTransaction`, y con transacciones reales el patrón Command deja de ser el único mecanismo de consistencia y pasa a cubrir sólo lo que la transacción no alcanza. **El I simplifica al II**: extraer las reglas de transición a objetos de estado achica los métodos del BLL, y sobre métodos chicos y homogéneos los decoradores quedan triviales. El capítulo 7 desarrolla el orden recomendado.

Transiciones de estado dispersas → State

Orden de producción, Venta, Orden de compra y Cotización.

3.1 Diagnóstico

El circuito de fábrica del RFN1 es una máquina de estados con seis estados y siete transiciones. Esa máquina existe en el sistema, pero no existe como *objeto*: está codificada tres veces en tres lenguajes distintos.

1. **En el BLL, como guardas.** Cada método de `OrdenProduccionBLL06AV` abre con un `if` que verifica el estado de partida y lanza `ValidacionException06AV`.
2. **En el BLL, como un `switch`.** `VolverAtras()` resuelve el estado anterior con un `switch` de seis ramas.
3. **En la UI, como una cascada de `if`.** `ComprasControl.EstadoDe()` —y sus equivalentes en `ProduccionControl` y `VentasControl`— deciden rótulo, color y "paso siguiente" repitiendo la misma información.

ANTES · BLL — la regla de transición como guarda y como switch

```
// — BLL/OrdenProduccionBLL06AV.cs — la regla de transición, dispersa —
public void AsignarLinea(int numeroOrden, int idLinea, DateTime fechaInicio, string responsable)
{
    var orden = ObtenerOrden0Excepcion(numeroOrden);
    if (orden.Estado != EstadoOrdenProduccion06AV.Pendiente) // ← regla 1
        throw new ValidacionException06AV("estado", "Solo se puede planificar una orden en estado
Pendiente.");
    ...
}

public void IniciarEnsamblaje(int numeroOrden)
{
    var orden = ObtenerOrden0Excepcion(numeroOrden);
    if (orden.Estado != EstadoOrdenProduccion06AV.Planificada) // ← regla 2
        throw new ValidacionException06AV("estado", "Solo se puede iniciar el ensamblaje de una orden
Planificada.");
    CambiarEstado(numeroOrden, EstadoOrdenProduccion06AV.EnEnsamblaje);
}

public string RegistrarControlCalidad(int numeroOrden, ControlCalidad06AV control)
{
    var orden = ObtenerOrden0Excepcion(numeroOrden);
    if (orden.Estado != EstadoOrdenProduccion06AV.EnEnsamblaje && // ← regla 3
        orden.Estado != EstadoOrdenProduccion06AV.EnRevision)
        throw new ValidacionException06AV("estado", "Solo se puede cerrar una orden En ensamblaje o En
revisión.");
    ...
}

public EstadoOrdenProduccion06AV VolverAtras(int numeroOrden)
{
    var orden = ObtenerOrden0Excepcion(numeroOrden);
    EstadoOrdenProduccion06AV anterior;

    switch (orden.Estado) // ← EL SWITCH: el olor canónico que pide State
    {
        case EstadoOrdenProduccion06AV.Planificada:
```

```

        anterior = EstadoOrdenProduccion06AV.Pendiente; break;
    case EstadoOrdenProduccion06AV.EnEnsamblaje:
        anterior = EstadoOrdenProduccion06AV.Planificada; break;
    case EstadoOrdenProduccion06AV.EnRevision:
        anterior = EstadoOrdenProduccion06AV.EnEnsamblaje; break;
    case EstadoOrdenProduccion06AV.Finalizada:
        throw new ValidacionException06AV("estado",
            "La orden ya está finalizada: el stock fue descontado y tiene N° de serie asignado.");
    case EstadoOrdenProduccion06AV.Entregada:
        throw new ValidacionException06AV("estado",
            "Una orden entregada ya tiene la factura emitida y no se puede volver atrás.");
    default:
        throw new ValidacionException06AV("estado", "La orden está en el primer paso; no hay un
paso anterior.");
    }
    ...
}

```

ANTES · UI — la misma máquina de estados, tercera copia

```

// — UI/Controles/ComprasControl.cs — la MISMA máquina de estados, tercera copia —
// Acá no se valida: se decide rótulo, color y "qué sigue". Pero la información
// que consulta es idéntica a la que el BLL usa para validar.
private void EstadoDe(OrdenCompra06AV oc, PedidoCotizacion06AV cot,
    out string texto, out Color color, out Paso paso)
{
    var t = GestorIdioma06AV.Instancia;

    if (oc.Estado == EstadoOrdenCompra06AV.Finalizada)
    { texto = t.Obtener("pcf_est_finalizada"); color = Tema.Exito;      paso = Paso.Finalizada;
return; }

    if (oc.Estado == EstadoOrdenCompra06AV.RecibidaParcial)
    { texto = t.Obtener("pcf_est_parcial");    color = Tema.Advertencia; paso = Paso.Recibir;
return; }

    if (oc.Estado == EstadoOrdenCompra06AV.Enviada)
    { texto = t.Obtener("pcf_est_enviada");    color = Tema.Acento;      paso = Paso.Recibir;
return; }

    if (cot != null && cot.Estado == EstadoCotizacion06AV.PorAprobar)
    { texto = t.Obtener("pcf_est_por_aprobar"); color = Tema.Primario;    paso = Paso.Aprobar;
return; }

    if (cot != null && cot.Estado == EstadoCotizacion06AV.Desaprobada)
    { texto = t.Obtener("pcf_est_rechazada");  color = Tema.Peligro;     paso = Paso.Cotizar;
return; }

    texto = t.Obtener("pcf_est_pendiente"); color = Tema.Advertencia; paso = Paso.Cotizar;
}

// Y más abajo, una CUARTA copia implícita: qué botón mostrar.
switch (paso)
{
    case Paso.Cotizar:      SeccionCotizar(cot); break;
    case Paso.Aprobar:      SeccionAprobar(cot); break;
    case Paso.Recibir:      SeccionRecibir();    break;
    case Paso.Finalizada: SeccionFinalizada(); break;
}

```

LA CONSECUENCIA CONCRETA

Agregar un estado —por ejemplo “*En espera de insumos*”, perfectamente plausible dado que el RFN2 puede dejar una orden de compra en `RecibidaParcial` — obliga a editar: el enum, cuatro guardas del BLL, el `switch` de `VolverAtras`, la cascada de la UI y el `switch` de secciones. **Seis puntos de edición, y el compilador no avisa si te olvidás de alguno**, porque todos compilan igual con el caso faltante.

3.2 Fundamento teórico

STATE — GOF, PATRÓN DE COMPORTAMIENTO

Intención (GoF, pág. 305): «Permitir que un objeto altere su comportamiento cuando cambia su estado interno. El objeto parecerá cambiar de clase.»

Aplicabilidad: el propio libro enuncia el criterio exacto de este caso — «las operaciones tienen sentencias condicionales grandes y de múltiples partes que dependen del estado del objeto [...] Este patrón coloca cada rama de la condicional en una clase separada».

Participantes: *Context* (mantiene la instancia del estado actual y delega en ella) · *State* (interfaz común) · *ConcreteState* (implementa el comportamiento asociado a un estado).

Consecuencias: ① localiza el comportamiento dependiente del estado y particiona el comportamiento por estados; ② hace explícitas las transiciones —dejan de ser asignaciones a una variable y pasan a ser un cambio de objeto, que es atómico; ③ los objetos de estado pueden compartirse si no tienen datos propios.

POR QUÉ STATE Y NO UN SIMPLE DICCIONARIO DE TRANSICIONES

Una tabla `Dictionary` de origen—evento—destino resolvería el `switch`, pero no el resto: no tiene dónde colgar el rótulo de idioma, la severidad visual, los permisos de la UI ni el motivo específico por el que un estado bloquea una operación. El objeto de estado sí: **concentra en un solo lugar todo lo que el sistema sabe sobre "estar en ese estado"**, y es eso lo que elimina las tres copias en simultáneo.

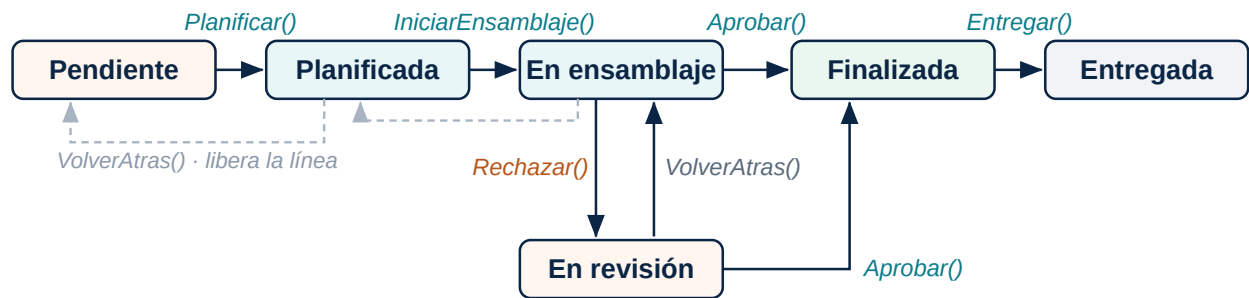
3.3 Diseño propuesto

Dos decisiones de diseño gobiernan toda la solución y son las que la vuelven aplicable sin romper nada:

1. **El enum no desaparece.** `EstadoOrdenProduccion06AV` sigue siendo el valor que persiste el MPP en la columna `Estado`. El objeto de estado se *deriva* del enum mediante una fábrica.

SIN MIGRACIÓN DE BASE DE DATOS

2. **El estado decide, el BLL ejecuta.** El objeto de estado responde a *qué estado se va*; los efectos laterales (persistir, liberar la línea, auditar) siguen siendo del BLL. Esto mantiene a `BE` sin dependencias hacia `MPP` y respeta la arquitectura en capas existente.



Finalizada y Entregada no tienen transiciones de salida: heredan el rechazo por defecto

Fig. 1 — Máquina de estados de la orden de producción (RFN1). Cada nodo pasa a ser una clase que implementa `IEstadoOrdenProduccion06AV`; cada flecha, un método que devuelve el estado destino. Las flechas grises punteadas son `VolverAtras()`. *Finalizada y Entregada sólo heredan el comportamiento por defecto de la clase base: rechazan todo, con su propio motivo.*

3.4 Implementación

NUEVO · BE/Estados/IEstadoOrdenProduccion06AV.cs

```
// =====
// BE/Estados/IEstadoOrdenProduccion06AV.cs      (NUEVO)
// El "State" de GoF. Cada estado concreto conoce:
//   · qué transiciones admite y a cuál estado llevan → reemplaza los if del BLL
//   · cómo se presenta (clave de idioma + severidad) → reemplaza los if de la UI
// Vive en BE porque es conocimiento del DOMINIO, no de la persistencia ni de
// la pantalla. BE sólo referencia SER, así que no arrastra dependencias nuevas.
// =====
namespace BE.Estados
{
    /// <summary>Severidad visual del estado. La UI mapea esto a un color del Tema;
    /// el dominio no conoce colores.</summary>
    public enum SeveridadEstado06AV { Neutro, EnCurso, Atencion, Exito, Bloqueado }

    public interface IEstadoOrdenProduccion06AV
    {
        // — Identidad y persistencia —
        /// <summary>Valor que se persiste. El enum NO desaparece: sigue siendo la
        /// representación en base de datos. El objeto de estado se DERIVA de él.</summary>
        EstadoOrdenProduccion06AV Valor { get; }

        // — Presentación (consumido por la UI) —
        string ClaveRotulo { get; } // "pcf_est_planificada"
        string ClaveAyudaSiguiente { get; } // "pcf_hint_op_ensamblar"
        SeveridadEstado06AV Severidad { get; }

        // — Consultas (la UI pregunta qué botones habilitar) —
        bool PuedeAsignarLinea { get; }
        bool PuedeIniciarEnsamblaje { get; }
        bool PuedeControlarCalidad { get; }
        bool PuedeVolverAtras { get; }

        // — Transiciones (el BLL delega la regla) —
        /// Devuelven el estado destino o lanzan TransicionInvalidaException06AV.
        IEstadoOrdenProduccion06AV Planificar();
        IEstadoOrdenProduccion06AV IniciarEnsamblaje();
        IEstadoOrdenProduccion06AV Aprobar(); // control de calidad OK
    }
}
```

```

        IEstadoOrdenProduccion06AV Rechazar();    // control de calidad NO OK
        IEstadoOrdenProduccion06AV Entregar();
        IEstadoOrdenProduccion06AV VolverAtras();
    }
}

```

La clase base es la pieza que elimina los `if`: **establece que la negativa es el comportamiento por defecto**. Un estado concreto sólo escribe lo que sí puede hacer; todo lo demás queda prohibido sin que nadie lo tenga que verificar.

NUEVO · BE/Estados/EstadoOrdenProduccionBase06AV.cs

```

// =====
// BE/Estados/EstadoOrdenProduccionBase06AV.cs      (NUEVO)
// Clase base con el comportamiento por DEFECTO: "esta transición no se puede".
// Cada estado concreto sólo sobrescribe lo que SÍ puede hacer.
// Esto es lo que elimina los if: la negativa es el default, no un chequeo.
// =====
namespace BE.Estados
{
    public abstract class EstadoOrdenProduccionBase06AV : IEstadoOrdenProduccion06AV
    {
        public abstract EstadoOrdenProduccion06AV Valor { get; }
        public abstract string ClaveRotulo { get; }
        public abstract string ClaveAyudaSiguiente { get; }
        public abstract SeveridadEstado06AV Severidad { get; }

        // Por defecto NADA está permitido. Cada estado concreto abre lo suyo.
        public virtual bool PuedeAsignarLinea      => false;
        public virtual bool PuedeIniciarEnsamblaje => false;
        public virtual bool PuedeControlarCalidad  => false;
        public virtual bool PuedeVolverAtras      => false;

        public virtual IEstadoOrdenProduccion06AV Planificar()      => Negar("planificar la orden");
        public virtual IEstadoOrdenProduccion06AV IniciarEnsamblaje()=> Negar("iniciar el ensamblaje");
        public virtual IEstadoOrdenProduccion06AV Aprobar()         => Negar("aprobar el control de
calidad");
        public virtual IEstadoOrdenProduccion06AV Rechazar()        => Negar("rechazar el control de
calidad");
        public virtual IEstadoOrdenProduccion06AV Entregar()        => Negar("entregar el equipo");
        public virtual IEstadoOrdenProduccion06AV VolverAtras()      => Negar("volver al paso
anterior");

        /// <summary>Motivo específico por el que ESTE estado bloquea la transición.
        /// Cada estado concreto puede reemplazarlo con su propia explicación.</summary>
        protected virtual string MotivoDeBloqueo => null;

        protected IEstadoOrdenProduccion06AV Negar(string accion)
        {
            throw new TransicionInvalidaException06AV(
                Valor, accion,
                MotivoDeBloqueo ?? $"una orden en estado '{Valor}' no admite esa operación");
        }

        public override string ToString() => Valor.ToString();
        public override bool Equals(object obj) =>
            obj is IEstadoOrdenProduccion06AV e && e.Valor == Valor;
        public override int GetHashCode() => (int)Valor;
    }
}

```

```
// =====
// BE/Estados/TransicionInvalidaException06AV.cs (NUEVO)
// Excepción de DOMINIO. No se usa ValidacionException06AV porque esa vive en
// BLL.Excepciones y BE no referencia BLL: invertir esa dependencia rompería
// la arquitectura en capas. El BLL la traduce en su frontera (ver §4.5).
// =====
namespace BE.Estados
{
    public class TransicionInvalidaException06AV : Exception
    {
        public TransicionInvalidaException06AV(EstadoOrdenProduccion06AV origen,
            string accion, string motivo)
            : base($"No se puede {accion}: {motivo}.")
        {
            EstadoOrigen = origen;
            Accion = accion;
        }

        public EstadoOrdenProduccion06AV EstadoOrigen { get; }
        public string Accion { get; }
    }
}
```

NUEVO · BE/Estados/EstadosOrdenProduccion06AV.cs – estados concretos

```
// =====
// BE/Estados/EstadosOrdenProduccion06AV.cs (NUEVO)
// Los seis estados concretos. Son SIN ESTADO PROPIO (inmutables), así que se
// comparten como instancias únicas: Flyweight + Singleton, igual que hace la
// implementación canónica de GoF (pág. 308: "State objects can often be shared").
// =====
namespace BE.Estados
{
    internal sealed class Pendiente06AV : EstadoOrdenProduccionBase06AV
    {
        public override EstadoOrdenProduccion06AV Valor => EstadoOrdenProduccion06AV.Pendiente;
        public override string ClaveRotulo => "pcf_est_op_pendiente";
        public override string ClaveAyudaSiguiente => "pcf_hint_op_planificar";
        public override SeveridadEstado06AV Severidad => SeveridadEstado06AV.Atencion;

        public override bool PuedeAsignarLinea => true;

        // Pendiente es el PRIMER paso: no hay anterior. Mensaje propio.
        protected override string MotivoDeBloqueo => "la orden está en el primer paso del circuito";

        public override IEstadoOrdenProduccion06AV Planificar()
            => EstadosOrdenProduccion06AV.Planificada;
    }

    internal sealed class Planificada06AV : EstadoOrdenProduccionBase06AV
    {
        public override EstadoOrdenProduccion06AV Valor => EstadoOrdenProduccion06AV.Planificada;
        public override string ClaveRotulo => "pcf_est_op_planificada";
        public override string ClaveAyudaSiguiente => "pcf_hint_op_ensamblar";
        public override SeveridadEstado06AV Severidad => SeveridadEstado06AV.EnCurso;

        public override bool PuedeIniciarEnsamblaje => true;
        public override bool PuedeVolverAtras => true;

        public override IEstadoOrdenProduccion06AV IniciarEnsamblaje()
            => EstadosOrdenProduccion06AV.EnEnsamblaje;
    }
}
```

```

        // Volver atrás desde Planificada implica DESPLANIFICAR y soltar la línea.
        // El estado dice a dónde se va; el efecto lateral lo ejecuta el BLL (§4.5).
        public override IEstadoOrdenProduccion06AV VolverAtras()
            => EstadosOrdenProduccion06AV.Pendiente;
    }

    internal sealed class EnEnsamblaje06AV : EstadoOrdenProduccionBase06AV
    {
        public override EstadoOrdenProduccion06AV Valor => EstadoOrdenProduccion06AV.EnEnsamblaje;
        public override string ClaveRotulo            => "pcf_est_op_ensamblaje";
        public override string ClaveAyudaSiguiete     => "pcf_hint_op_cerrar";
        public override SeveridadEstado06AV Severidad => SeveridadEstado06AV.EnCurso;

        public override bool PuedeControlarCalidad => true;
        public override bool PuedeVolverAtras     => true;

        public override IEstadoOrdenProduccion06AV Aprobar() => EstadosOrdenProduccion06AV.Finalizada;
        public override IEstadoOrdenProduccion06AV Rechazar() => EstadosOrdenProduccion06AV.EnRevision;
        public override IEstadoOrdenProduccion06AV VolverAtras() =>
            EstadosOrdenProduccion06AV.Planificada;
    }

    internal sealed class EnRevision06AV : EstadoOrdenProduccionBase06AV
    {
        public override EstadoOrdenProduccion06AV Valor => EstadoOrdenProduccion06AV.EnRevision;
        public override string ClaveRotulo            => "pcf_est_op_revision";
        public override string ClaveAyudaSiguiete     => "pcf_hint_op_revision";
        public override SeveridadEstado06AV Severidad => SeveridadEstado06AV.Atencion;

        public override bool PuedeControlarCalidad => true;    // se vuelve a controlar
        public override bool PuedeVolverAtras     => true;

        public override IEstadoOrdenProduccion06AV Aprobar() => EstadosOrdenProduccion06AV.Finalizada;
        public override IEstadoOrdenProduccion06AV Rechazar() => this;    // sigue en revisión
        public override IEstadoOrdenProduccion06AV VolverAtras() =>
            EstadosOrdenProduccion06AV.EnEnsamblaje;
    }

    internal sealed class Finalizada06AV : EstadoOrdenProduccionBase06AV
    {
        public override EstadoOrdenProduccion06AV Valor => EstadoOrdenProduccion06AV.Finalizada;
        public override string ClaveRotulo            => "pcf_est_op_finalizada";
        public override string ClaveAyudaSiguiete     => "pcf_hint_op_entregar";
        public override SeveridadEstado06AV Severidad => SeveridadEstado06AV.Exito;

        protected override string MotivoDeBloqueo =>
            "la orden ya está finalizada: el stock fue descontado y tiene N° de serie asignado";

        public override IEstadoOrdenProduccion06AV Entregar() => EstadosOrdenProduccion06AV.Entregada;
    }

    internal sealed class Entregada06AV : EstadoOrdenProduccionBase06AV
    {
        public override EstadoOrdenProduccion06AV Valor => EstadoOrdenProduccion06AV.Entregada;
        public override string ClaveRotulo            => "pcf_est_op_entregada";
        public override string ClaveAyudaSiguiete     => "pcf_hint_op_entregada";
        public override SeveridadEstado06AV Severidad => SeveridadEstado06AV.Bloqueado;

        protected override string MotivoDeBloqueo =>
            "una orden entregada ya tiene la factura emitida";
    }
}

```

```
// =====
// BE/Estados/EstadosOrdenProduccion06AV.cs (cont.) - FÁBRICA + INSTANCIAS
// Traduce el valor persistido (int del enum) al objeto de estado.
// Es el único punto del sistema que conoce el mapeo enum ↔ objeto.
// =====

namespace BE.Estados
{
    public static class EstadosOrdenProduccion06AV
    {
        // Instancias únicas: los estados no tienen datos propios (Flyweight).
        public static readonly IEstadoOrdenProduccion06AV Pendiente = new Pendiente06AV();
        public static readonly IEstadoOrdenProduccion06AV Planificada = new Planificada06AV();
        public static readonly IEstadoOrdenProduccion06AV EnEnsamblaje = new EnEnsamblaje06AV();
        public static readonly IEstadoOrdenProduccion06AV EnRevision = new EnRevision06AV();
        public static readonly IEstadoOrdenProduccion06AV Finalizada = new Finalizada06AV();
        public static readonly IEstadoOrdenProduccion06AV Entregada = new Entregada06AV();

        private static readonly Dictionary<EstadoOrdenProduccion06AV, IEstadoOrdenProduccion06AV> _mapa
        =
            new Dictionary<EstadoOrdenProduccion06AV, IEstadoOrdenProduccion06AV>
            {
                { EstadoOrdenProduccion06AV.Pendiente, Pendiente },
                { EstadoOrdenProduccion06AV.Planificada, Planificada },
                { EstadoOrdenProduccion06AV.EnEnsamblaje, EnEnsamblaje },
                { EstadoOrdenProduccion06AV.EnRevision, EnRevision },
                { EstadoOrdenProduccion06AV.Finalizada, Finalizada },
                { EstadoOrdenProduccion06AV.Entregada, Entregada },
            };

        public static IEstadoOrdenProduccion06AV Desde(EstadoOrdenProduccion06AV valor)
        {
            IEstadoOrdenProduccion06AV estado;
            if (!_mapa.TryGetValue(valor, out estado))
                throw new ArgumentOutOfRangeException(nameof(valor),
                    $"No hay un objeto de estado definido para '{valor}'. " +
                    "Si agregaste un estado al enum, registralo acá.");
            return estado;
        }
    }
}

// =====
// BE/OrdenProduccion06AV.cs (MODIFICADO - dos líneas)
// La entidad es el "Context" de GoF. NO se toca la propiedad Estado: sigue
// siendo el enum que persiste el MPP. El objeto de estado se DERIVA.
// Esta decisión es la que hace que la migración no toque la base de datos.
// =====

public class OrdenProduccion06AV
{
    public EstadoOrdenProduccion06AV Estado { get; set; } = EstadosOrdenProduccion06AV.Pendiente;

    /// <summary>Comportamiento asociado al estado actual (patrón State).</summary>
    public IEstadoOrdenProduccion06AV EstadoActual => EstadosOrdenProduccion06AV.Desde(Estado);

    /// <summary>Aplica una transición ya resuelta por el objeto de estado.</summary>
    public void TransicionarA(IEstadoOrdenProduccion06AV nuevo) { Estado = nuevo.Valor; }

    // ... el resto de la clase queda igual
}
```

3.5 El BLL después

Comparar este bloque con el de §3.1 es el argumento completo del patrón: las guardas de estado desaparecen, el `switch` desaparece, y lo que queda es exactamente lo que le compete al BLL —coordinar persistencia, efectos laterales y auditoría.

DESPUÉS · BLL/OrdenProduccionBLL06AV.cs

```
// =====
// BLL/OrdenProduccionBLL06AV.cs      (DESPUÉS)
// El BLL ya no decide si la transición es válida: PREGUNTA. Queda sólo con
// lo que de verdad le corresponde – coordinar MPP, efectos laterales y auditoría.
// =====
public void AsignarLinea(int numeroOrden, int idLinea, DateTime fechaInicio, string responsable)
{
    var orden = ObtenerOrden0Excepcion(numeroOrden);

    // Una línea reemplaza el if de estado. Si no se puede, el propio estado explica por qué.
    var destino = Transicion(() => orden.EstadoActual.Planificar());

    if (string.IsNullOrEmpty(responsable))
        throw new ValidacionException06AV("responsable", "El responsable técnico es obligatorio.");

    var linea = _lineasMpp.ObtenerPorId(idLinea);
    if (linea == null) throw new NoEncontradoException06AV($"No existe la línea #{idLinea}.");
    if (!linea.Disponible)
        throw new ValidacionException06AV("linea", "La línea elegida no está disponible.");

    try
    {
        _mpp.Planificar(numeroOrden, idLinea, fechaInicio, responsable);
        linea.Disponible = false;
        _lineasMpp.Modificar(linea);
    }
    catch (Exception ex) { throw new AccesoDatosException06AV("No se pudo planificar la orden.", ex); }

    orden.TransicionarA(destino);
    AuditoriaPcFactory06AV.Modificacion(
        $"Orden #{numeroOrden} planificada (línea #{idLinea})", ModuloBitacora.Produccion);
}

public void IniciarEnsamblaje(int numeroOrden)
{
    var orden = ObtenerOrden0Excepcion(numeroOrden);
    var destino = Transicion(() => orden.EstadoActual.IniciarEnsamblaje());
    CambiarEstado(numeroOrden, destino.Valor);
}

public EstadoOrdenProduccion06AV VolverAtras(int numeroOrden)
{
    var orden = ObtenerOrden0Excepcion(numeroOrden);
    var anterior = Transicion(() => orden.EstadoActual.VolverAtras()); // ← adiós al switch

    try
    {
        // El efecto lateral sigue siendo responsabilidad del BLL, no del estado:
        // desplanificar toca la persistencia y libera un recurso compartido.
        if (anterior.Valor == EstadoOrdenProduccion06AV.Pendiente)
        {
            _mpp.Desplanificar(numeroOrden);
            LiberarLinea(orden);
        }
    }
}
```

```

        else _mpp.CambiarEstado(numeroOrden, anterior.Valor);
    }
    catch (Exception ex) { throw new AccesoDatosException06AV("No se pudo volver al paso anterior.",
ex); }

    orden.TransicionarA(anterior);
    AuditoriaPcFactory06AV.Modificacion(
        $"Orden #{numeroOrden} vuelta a estado {anterior.Valor}", ModuloBitacora.Produccion);
    return anterior.Valor;
}

// — Traducción en la frontera de capa —————
// BE no puede lanzar ValidacionException06AV (vive en BLL). Este adaptador de
// una línea preserva el contrato de excepciones que la UI ya conoce: no hay
// que tocar un solo catch de las pantallas existentes.
private static IEstadoOrdenProduccion06AV Transicion(Func<IEstadoOrdenProduccion06AV> transicion)
{
    try { return transicion(); }
    catch (TransicionInvalidaException06AV ex)
    { throw new ValidacionException06AV("estado", ex.Message); }
}

```

El método `Transicion()` de tres líneas merece atención: es un *adaptador de frontera*. Traduce la excepción de dominio (`TransicionInvalidaException06AV`, que vive en BE) a la excepción que la UI ya sabe capturar (`ValidacionException06AV`, que vive en BLL). Gracias a él, **ninguna pantalla existente necesita cambiar su manejo de errores**.

DESPUÉS · UI — la cascada de if desaparece

```
// =====
// UI/Controles/ProduccionControl.cs      (DESPUÉS)
// La cascada de if desaparece. La UI no sabe qué estados existen: le pregunta
// al estado cómo se llama, qué severidad tiene y qué se puede hacer con él.
// Agregar un estado nuevo al circuito NO obliga a tocar esta pantalla.
// =====

private void ActualizarDetalle(OrdenProduccion06AV orden)
{
    var t = GestorIdioma06AV.Instancia;
    var e = orden.EstadoActual;

    lblDetEstado.Text      = "● " + t.Obtener(e.ClaveRotulo);
    lblDetEstado.ForeColor = Tema.PorSeveridad(e.Severidad);
    lblHint.Text           = t.Obtener(e.ClaveAyudaSiguiente);

    btnAsignarLinea.Visible = e.PuedeAsignarLinea;
    btnEnsamblar.Visible   = e.PuedeIniciarEnsamblaje;
    btnCerrar.Visible      = e.PuedeControlarCalidad;
    btnVolverAtras.Visible = e.PuedeVolverAtras;
}

// — UI/Tema.cs — único punto donde severidad de dominio → color de marca —
public static Color PorSeveridad(SeveridadEstado06AV s)
{
    switch (s)
    {
        case SeveridadEstado06AV.Exito:      return Exitos;
        case SeveridadEstado06AV.EnCurso:    return Primario;
        case SeveridadEstado06AV.Atencion:   return Advertencia;
        case SeveridadEstado06AV.Bloqueado:  return TextoSuave;
        default:                             return Texto;
    }
}
}
```

3.6 Migración y verificación

Paso	Acción	Cómo verificar que no rompiste nada
1	Crear <code>BE/Estados/</code> con la interfaz, la base, los seis estados y la fábrica. No tocar nada más.	Compila. El sistema sigue funcionando exactamente igual: nadie usa las clases nuevas todavía.
2	Agregar <code>EstadoActual</code> y <code>TransicionarA</code> a <code>OrdenProduccion06AV</code> .	Compila. Verificar en el depurador que <code>orden.EstadoActual.Valor == orden.Estado</code> para una orden de cada estado.
3	Migrar un solo método del BLL (sugerido: <code>IniciarEnsamblaje</code> , el más simple). Dejar el resto con sus <code>if</code> .	Probar el caso feliz y el caso de error en la pantalla de Producción. El mensaje de error debe seguir apareciendo en el mismo diálogo.
4	Migrar los métodos restantes, incluido <code>VolverAtras</code> .	Recorrer el circuito completo del RFN1: crear orden → asignar línea → ensamblar → control OK → entregar. Y el alternativo: control rechazado → En revisión → control OK.
5	Migrar la UI a <code>EstadoActual</code> , agregar <code>Tema.PorSeveridad</code> y las claves de idioma en los tres JSON.	Cambiar de idioma con una orden en cada estado y verificar rótulos. Cambiar de tema claro/oscuro y verificar colores.
6	Repetir el ciclo para <code>EstadoOrdenCompra06AV</code> y <code>EstadoCotizacion06AV</code> (RFN2).	Circuito completo del RFN2, incluida la recepción parcial.

RESULTADO ESPERADO

Agregar el estado *"En espera de insumos"* pasa de editar seis puntos a **escribir una clase nueva y registrarla en la fábrica**. La UI no se toca: pregunta por `ClaveRotulo`, `Severidad` y `Puede...`, y el estado nuevo los responde igual que los demás. Eso es OCP cumplido, verificable.

Seguridad y auditoría por convención → Decorator / Proxy

Autorización por patentes, bitácora y dígito verificador.

4.1 Diagnóstico

Este es el dolor con mayor impacto real, porque no es sólo deuda técnica: es un **problema de seguridad y de integridad de datos** documentado en el propio repositorio.

El sistema define su autorización por *patentes* (Composite: Rol → Familias → Patentes), y

`UsuarioSesion06AV.TienePermiso()` resuelve la consulta correctamente. El problema es **dónde se la invoca**: existe un método `ExigirPatente`, declarado `private static`, en `CompraInsumosBLL06AV` — y sólo ahí. Las otras cinco clases de negocio (`VentasBLL06AV`, `OrdenProduccionBLL06AV`, `EntregasBLL06AV`, `ComponentesBLL06AV`, `ClientesBLL06AV`) no verifican ninguna patente.

IMPLICANCIA

La autorización efectiva del 83% de las operaciones de negocio vive en el **filtrado del menú lateral** de `FRMMain`, que es control de presentación. Un usuario sin la patente `GestionarVentas` no ve el botón, pero si la capa de negocio se invoca por cualquier otra vía —otra pantalla, un atajo, una versión futura del código— no hay nada que lo detenga. **En términos de diseño: la regla de seguridad está en la capa equivocada.**

El segundo problema es la integridad. El `AGENTS.md` del repositorio lo enuncia como advertencia explícita: «Después de cada escritura, `IntegridadBLL06AV.RecalcularSeguro()` recalcula los checksums. Si una escritura saltea el registro de auditoría, la línea base del DV se desincroniza y los logins detectan corrupción falsa. Siempre pasar por el BLL — nunca saltar.» Eso describe con precisión una **convención que el compilador no verifica**: el recálculo del dígito verificador se dispara únicamente desde `AuditoriaPcFactory06AV.Registrar()`, y esa llamada se escribe a mano, método por método.

ANTES · El trío repetido, y el BLL que no lo repite

```
// — BLL/CompraInsumosBLL06AV.cs — el ÚNICO BLL que verifica patentes —
private static void ExigirPatente(PatenteEnum06AV patente, string accion)
{
    var usuario = UsuarioSesion06AV.Instancia().UsuarioActual;
    if (usuario == null)
        throw new ValidacionException06AV("usuario", $"Se requiere un usuario autenticado para {accion}.");
    if (!UsuarioSesion06AV.Instancia().TienePermiso(patente))
        throw new ValidacionException06AV("patente",
            $"El usuario '{usuario.Login}' no tiene el permiso '{patente}' requerido para {accion}.");
}

public OrdenCompra06AV RegistrarOrdenCompra(List<DetalleComponente06AV> faltantes,
    DateTime fechaLimite, Usuario06AV repositor)
{
    ExigirPatente(PatenteEnum06AV.RegistrarOrdenCompra, "registrar una orden de compra"); // ← a mano
    ... 40 líneas de validación y lógica ...
    AuditoriaPcFactory06AV.Alta($"Orden de compra #{oc.NumeroCompra}", ModuloBitacora.Compras); // ← a mano
    return oc;
}

// — BLL/VentasBLL06AV.cs — MISMA operación crítica, SIN ExigirPatente —
public Venta06AV RegistrarVenta(Cliente06AV cliente, Computadora06AV computadora, DateTime fecha)
{
    // (no hay control de patente: la autorización vive sólo en el menú de FRMMain)
    ... lógica ...
    AuditoriaPcFactory06AV.Alta($"Venta #{venta.NumeroVenta}", ModuloBitacora.Ventas);
    return venta;
}

// — BLL/AuditoriaPcFactory06AV.cs — el acoplamiento oculto —
private static void Registrar(Action<BitacoraBLL06AV> accion)
{
    try { new IntegridadBLL06AV().RecalcularSeguro(); } catch { } // ← el DV depende de ESTO
    try { accion(new BitacoraBLL06AV()); } catch { }
}

```

4.2 Fundamento teórico

DECORATOR — GOF, PATRÓN ESTRUCTURAL

Intención (GoF, pág. 175): «Asignar responsabilidades adicionales a un objeto dinámicamente. Los decoradores ofrecen una alternativa flexible a la herencia para extender funcionalidad.»

Aplicabilidad: «para agregar responsabilidades a objetos individuales de forma dinámica y transparente, esto es, sin afectar a otros objetos».

Estructura: el decorador *implementa la misma interfaz* que el componente que envuelve y mantiene una referencia a él. Esa identidad de interfaz es lo que permite apilarlos y lo que hace que el cliente no note la diferencia.

PROXY DE PROTECCIÓN — GOF, PATRÓN ESTRUCTURAL

Intención (GoF, pág. 207): «Proporcionar un sustituto o representante de otro objeto para controlar el acceso a él.» El libro distingue variantes; la que aplica acá es el *protection proxy*, que «controla el acceso al objeto original», y es literalmente el rol del decorador de autorización.

DECORATOR O PROXY: LA DISTINCIÓN, Y POR QUÉ ACÁ SE USAN LOS DOS

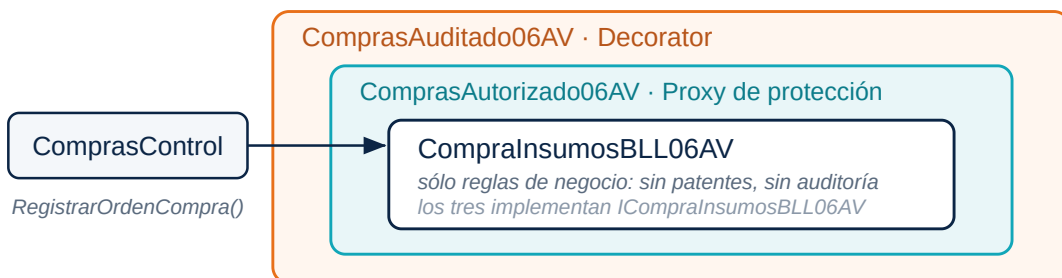
Estructuralmente son idénticos —ambos envuelven un objeto del mismo tipo. La diferencia es de **intención**: el Proxy controla el acceso, el Decorator agrega *responsabilidades*. En esta solución hay un envoltorio de cada tipo, apilados:

- **Proxy de protección** — verifica la patente antes de dejar pasar la llamada.
- **Decorator** — agrega el registro en bitácora y el recálculo del DV después de que la llamada tuvo éxito.

Nombrarlos correctamente no es un detalle académico: el orden en que se apilan tiene consecuencias funcionales, y se discute en §4.5.

4.3 Diseño propuesto

El paso cero es inevitable: **para decorar hace falta una interfaz**. Sin un tipo común entre el componente y el envoltorio el patrón no se puede aplicar. Extraer la interfaz de cada BLL no cambia el cuerpo de las clases existentes —sólo agrega la declaración— y es, además, lo que rompe el acoplamiento actual de la UI, que hoy hace `new CompraInsumosBLL06AV()` directamente (violación del Principio de Inversión de Dependencias).



① exige patente → ② aplica reglas → ③ audita y recalcula el dígito verificador
Un intento sin permiso no llega al núcleo ni se audita: la excepción del proxy corta antes.

Fig. 2 — Composición de envoltorios. La UI depende de la interfaz y recibe la pila ya armada por `FabricaServicios06AV`; no sabe cuántas capas hay ni en qué orden. Cambiar la política de seguridad de todo el sistema es cambiar un renglón de esa fábrica.

4.4 Implementación

NUEVO · BLL/Contratos/ICompraInsumosBLL06AV.cs

```
// =====
// BLL/Contratos/ICompraInsumosBLL06AV.cs          (NUEVO)
// Paso 0 indispensable: para decorar hace falta una INTERFAZ. Sin ella no hay
// tipo común entre el componente y el decorador, y el patrón no se puede aplicar.
// La clase existente pasa a implementarla sin cambiar una línea de su cuerpo.
// =====
```

```

namespace BLL.Contratos
{
    public interface ICompraInsumosBLL06AV
    {
        // Lecturas
        List<Componente06AV> ObtenerFaltantes();
        List<OrdenCompra06AV> ObtenerOrdenesCompra();
        List<PedidoCotizacion06AV> ObtenerCotizaciones();
        List<PedidoCotizacion06AV> ObtenerCotizacionesPorOrden(string idOrdenCompra);
        List<DetalleComponente06AV> ObtenerPendienteDeRecibir(string idOrdenCompra);

        // Escrituras (las que exigen patente y se auditan)
        OrdenCompra06AV RegistrarOrdenCompra(List<DetalleComponente06AV> faltantes,
                                                DateTime fechaLimite, Usuario06AV repositor);
        PedidoCotizacion06AV RegistrarCotizacion(string idOrdenCompra, int idProveedor,
                                                  decimal costo, string condiciones);
        void AprobarCotizacion(string numeroCotizacion, Usuario06AV gerente);
        void DesaprobarCotizacion(string numeroCotizacion, Usuario06AV gerente);
        FacturaCompra06AV RegistrarFacturaCompra(string idOrdenCompra, DateTime fechaEntrega,
                                                  string observaciones,
                                                  List<DetalleComponente06AV> recibidos = null);
    }
}

// La clase de negocio queda LIMPIA: sin ExigirPatente, sin llamadas a Auditoría.
// Sólo reglas de negocio. Esto es Responsabilidad Única de verdad.
public class CompraInsumosBLL06AV : ICompraInsumosBLL06AV
{
    public OrdenCompra06AV RegistrarOrdenCompra(List<DetalleComponente06AV> faltantes,
                                                DateTime fechaLimite, Usuario06AV repositor)
    {
        // ← se borró la línea ExigirPatente(...)
        if (repositor == null) throw new ValidacionException06AV(...);
        ... reglas de negocio puras ...
        // ← se borró la línea AuditoriaPcFactory06AV.Alta(...)
        return oc;
    }
}

```

NUEVO · Proxy de protección + servicio de autorización

```

// =====
// BLL/Decoradores/ComprasAutorizado06AV.cs      (NUEVO)
// DECORADOR 1 – Autorización. Añade el control de patentes SIN tocar el núcleo.
// Es el "Proxy de protección" de GoF: mismo tipo, misma firma, controla el acceso.
// =====
namespace BLL.Decoradores
{
    public class ComprasAutorizado06AV : ICompraInsumosBLL06AV
    {
        private readonly ICompraInsumosBLL06AV _interno;    // ← el componente envuelto

        public ComprasAutorizado06AV(ICompraInsumosBLL06AV interno)
        {
            _interno = interno ?? throw new ArgumentNullException(nameof(interno));
        }

        // — Escrituras: se exige patente ANTES de delegar —
        public OrdenCompra06AV RegistrarOrdenCompra(List<DetalleComponente06AV> faltantes,
                                                    DateTime fechaLimite, Usuario06AV repositor)
        {
            Autorizacion06AV.Exigir(PatenteEnum06AV.RegistrarOrdenCompra, "registrar una orden de

```

```

        compra");
        return _interno.RegistrarOrdenCompra(faltantes, fechaLimite, repositor);
    }

    public PedidoCotizacion06AV RegistrarCotizacion(string idOrdenCompra, int idProveedor,
        decimal costo, string condiciones)
    {
        Autorizacion06AV.Exigir(PatenteEnum06AV.GestionarCotizaciones, "cargar una cotización");
        return _interno.RegistrarCotizacion(idOrdenCompra, idProveedor, costo, condiciones);
    }

    public void AprobarCotizacion(string numeroCotizacion, Usuario06AV gerente)
    {
        Autorizacion06AV.Exigir(PatenteEnum06AV.AprobarCotizacion, "aprobar una cotización");
        _interno.AprobarCotizacion(numeroCotizacion, gerente);
    }

    public FacturaCompra06AV RegistrarFacturaCompra(string idOrdenCompra, DateTime fechaEntrega,
        string observaciones,
        List<DetalleComponente06AV> recibidos = null)
    {
        Autorizacion06AV.Exigir(PatenteEnum06AV.GestionarCompras, "recibir insumos");
        return _interno.RegistrarFacturaCompra(idOrdenCompra, fechaEntrega, observaciones,
recibidos);
    }

    // — Lecturas: delegación pura, sin agregar comportamiento —
    public List<Componente06AV> ObtenerFaltantes() => _interno.ObtenerFaltantes();
    public List<OrdenCompra06AV> ObtenerOrdenesCompra() => _interno.ObtenerOrdenesCompra();
    public List<PedidoCotizacion06AV> ObtenerCotizaciones() => _interno.ObtenerCotizaciones();
    public List<PedidoCotizacion06AV> ObtenerCotizacionesPorOrden(string id)
        => _interno.ObtenerCotizacionesPorOrden(id);
    public List<DetalleComponente06AV> ObtenerPendienteDeRecibir(string id)
        => _interno.ObtenerPendienteDeRecibir(id);

    public void DesaprobarCotizacion(string numeroCotizacion, Usuario06AV gerente)
    {
        Autorizacion06AV.Exigir(PatenteEnum06AV.AprobarCotizacion, "desaprobar una cotización");
        _interno.DesaprobarCotizacion(numeroCotizacion, gerente);
    }
}

// =====
// SER/Seguridad/Autorizacion06AV.cs (NUEVO)
// El chequeo de patente, UNA sola vez en todo el sistema. Se mueve a SER
// porque es un servicio transversal, no lógica de Compras.
// =====
namespace SER.Seguridad
{
    public static class Autorizacion06AV
    {
        public static void Exigir(PatenteEnum06AV patente, string accion)
        {
            var sesion = UsuarioSesion06AV.Instancia();
            var usuario = sesion.UsuarioActual;

            if (usuario == null)
                throw new AutorizacionException06AV($"Se requiere un usuario autenticado para
{accion}.");

            if (!sesion.TienePermiso(patente))
                throw new AutorizacionException06AV(
                    $"El usuario '{usuario.Login}' no tiene el permiso '{patente}' requerido para

```

```

{accion}."");
    }

    /// <summary>Consulta sin excepción, para que la UI habilite o no un botón.</summary>
    public static bool Puede(PatenteEnum06AV patente)
        => UsuarioSesion06AV.Instancia().UsuarioActual != null
        && UsuarioSesion06AV.Instancia().TienePermiso(patente);
    }
}

```

NUEVO · Decorator de auditoría y punto de composición

```

// =====
// BLL/Decoradores/ComprasAuditado06AV.cs          (NUEVO)
// DECORADOR 2 – Auditoría + recálculo del dígito verificador.
// Acá está el valor real: la auditoría deja de ser una convención que el
// programador tiene que recordar y pasa a ser una propiedad de la composición.
// =====
namespace BLL.Decoradores
{
    public class ComprasAuditado06AV : ICompraInsumosBLL06AV
    {
        private readonly ICompraInsumosBLL06AV _interno;
        public ComprasAuditado06AV(ICompraInsumosBLL06AV interno) { _interno = interno; }

        public OrdenCompra06AV RegistrarOrdenCompra(List<DetalleComponente06AV> faltantes,
            DateTime fechaLimite, Usuario06AV repositor)
        {
            var oc = _interno.RegistrarOrdenCompra(faltantes, fechaLimite, repositor);
            // Se audita DESPUÉS y sólo si no hubo excepción: nunca se registra
            // como hecho algo que en realidad falló.
            AuditoriaPcFactory06AV.Alta(
                $"Orden de compra #{oc.NumeroCompra} (repositor {repositor?.Login})",
                ModuloBitacora.Compras);
            return oc;
        }

        public void AprobarCotizacion(string numeroCotizacion, Usuario06AV gerente)
        {
            _interno.AprobarCotizacion(numeroCotizacion, gerente);
            AuditoriaPcFactory06AV.Modificacion(
                $"Cotización {numeroCotizacion} aprobada por {gerente?.Login}",
                ModuloBitacora.Compras);
        }

        public FacturaCompra06AV RegistrarFacturaCompra(string idOrdenCompra, DateTime fechaEntrega,
            string observaciones,
            List<DetalleComponente06AV> recibidos = null)
        {
            var f = _interno.RegistrarFacturaCompra(idOrdenCompra, fechaEntrega, observaciones,
                recibidos);
            AuditoriaPcFactory06AV.Alta(
                $"Factura de compra {f.NumeroFactura} (OC {idOrdenCompra})", ModuloBitacora.Compras);
            return f;
        }

        // Lecturas: delegación pura. No se audita leer.
        public List<Componente06AV> ObtenerFaltantes() => _interno.ObtenerFaltantes();
        // ... resto de lecturas igual ...
    }
}

```

```
// =====
// BLL/FabricaServicios06AV.cs (NUEVO)
// El punto de composición. Es lo único que la UI conoce: nunca vuelve a hacer
// "new CompraInsumosBLL06AV()". Cambiar la política de seguridad o de auditoría
// del sistema entero es cambiar ESTE archivo.
// (Es un Factory Method; con un contenedor de inyección sería el registro del
// contenedor. Para un proyecto WinForms sin DI, esta fábrica alcanza y sobra.)
// =====
namespace BLL
{
    public static class FabricaServicios06AV
    {
        public static ICompraInsumosBLL06AV Compras()
        {
            => new ComprasAuditado06AV( // ③ audita el resultado
                new ComprasAutorizado06AV( // ② controla el permiso
                    new CompraInsumosBLL06AV() // ① aplica las reglas
                ));

            public static IVentasBLL06AV Ventas()
            {
                => new VentasAuditado06AV(new VentasAutorizado06AV(new VentasBLL06AV()));

            public static IOrdenProduccionBLL06AV Produccion()
            {
                => new ProduccionAuditado06AV(new ProduccionAutorizado06AV(new OrdenProduccionBLL06AV()));
            }
        }
    }

    // — UI: el único cambio que ven las pantallas —
    public partial class ComprasControl : UserControl
    {
        // ANTES: private readonly CompraInsumosBLL06AV _bll = new CompraInsumosBLL06AV();
        private readonly ICompraInsumosBLL06AV _bll = FabricaServicios06AV.Compras();
        // ... el resto del control no cambia una línea ...
    }
}
```

4.5 Decisiones de diseño a justificar en la defensa

EL ORDEN DE APILADO

La fábrica compone `Auditado(Autorizado(núcleo))`. Leído de afuera hacia adentro: la auditoría es la capa externa, la autorización la intermedia. Eso significa que **un intento no autorizado no se audita**, porque la excepción del proxy se propaga antes de que el decorador de auditoría llegue a registrar nada.

Si la política de seguridad exigiera registrar los intentos denegados —requisito razonable en un sistema con bitácora— hay dos salidas: invertir el orden a `Autorizado(Auditado(núcleo))`, o agregar un tercer envoltorio `ComprasAuditadoDeAccesos06AV` como capa más externa. **Que esta decisión sea un renglón en la fábrica, y no una modificación en veinticinco métodos, es exactamente el beneficio del patrón.**

POR QUÉ LA AUDITORÍA VA DESPUÉS Y NO ANTES

El decorador invoca primero `_interno.Método(...)` y audita con el resultado en la mano. Dos razones: nunca se registra como hecho algo que lanzó excepción, y el texto de la bitácora puede incluir datos que sólo existen después de ejecutar —el `NumeroCompra` que asigna la secuencia de la base, el `NumeroFactura` generado.

COSTO DEL PATRÓN, Y LA VARIANTE QUE LO ELIMINA

El decorador explícito tiene un costo honesto: hay que escribir todos los métodos de la interfaz aunque la mayoría sólo delegue. Con seis servicios de unos diez métodos y dos envoltorios cada uno, son del orden de 120 métodos de delegación. Es código trivial pero es ruido.

La alternativa es un **proxy dinámico**: `System.Reflection.DispatchProxy` está disponible en .NET Framework 4.7.1 y superior (vía paquete NuGet), genera el envoltorio en tiempo de ejecución y resuelve qué patente exigir leyendo un atributo declarado sobre la interfaz. Un solo archivo cubre todo el sistema.

VARIANTE · Proxy dinámico con DispatchProxy

```
// =====
// VARIANTE AVANZADA – Proxy dinámico con DispatchProxy
// Costo del decorador explícito: hay que escribir los N métodos de la interfaz,
// aunque la mayoría sólo delegue. Con 6 servicios × ~10 métodos × 2 decoradores
// son ~120 métodos de delegación.
//
// DispatchProxy (System.Reflection.DispatchProxy, disponible en .NET Framework
// 4.7.1+ vía NuGet) genera el proxy en tiempo de ejecución y resuelve la patente
// leyendo un atributo del método. Cero delegaciones a mano.
// =====

// 1) El atributo declarativo, sobre la INTERFAZ
namespace BLL.Contratos
{
    [AttributeUsage(AttributeTargets.Method)]
    public sealed class ExigePatenteAttribute06AV : Attribute
    {
        public ExigePatenteAttribute06AV(PatenteEnum06AV patente, string accion)
        { Patente = patente; Accion = accion; }

        public PatenteEnum06AV Patente { get; }
        public string Accion { get; }
    }

    [AttributeUsage(AttributeTargets.Method)]
    public sealed class AuditaAttribute06AV : Attribute
    {
        public AuditaAttribute06AV(ModuloBitacora modulo, TipoEventoAuditoria06AV tipo)
        { Modulo = modulo; Tipo = tipo; }

        public ModuloBitacora Modulo { get; }
        public TipoEventoAuditoria06AV Tipo { get; }
    }

    public interface ICompraInsumosBLL06AV
    {
        [ExigePatente06AV(PatenteEnum06AV.RegistrarOrdenCompra, "registrar una orden de compra")]
        [Audita06AV(ModuloBitacora.Compras, TipoEventoAuditoria06AV.Alta)]
        OrdenCompra06AV RegistrarOrdenCompra(List<DetalleComponente06AV> f, DateTime fl, Usuario06AV
r);

        [ExigePatente06AV(PatenteEnum06AV.AprobarCotizacion, "aprobar una cotización")]
        [Audita06AV(ModuloBitacora.Compras, TipoEventoAuditoria06AV.Modificacion)]
        void AprobarCotizacion(string numeroCotizacion, Usuario06AV gerente);

        List<Componente06AV> ObtenerFaltantes(); // sin atributos: no exige ni audita
    }
}

// 2) El proxy: UN solo archivo para TODO el sistema
namespace BLL.Decoradores
```

```

{
    public class InterceptorSeguridad06AV<T> : DispatchProxy where T : class
    {
        private T _interno;

        public static T Envolver(T interno)
        {
            var proxy = Create<T, InterceptorSeguridad06AV<T>>() as InterceptorSeguridad06AV<T>;
            proxy._interno = interno;
            return proxy as T;
        }

        protected override object Invoke(MethodInfo metodo, object[] argumentos)
        {
            // — Antes: autorización —————
            var permiso = metodo.GetCustomAttribute<ExigePatenteAttribute06AV>();
            if (permiso != null)
                Autorizacion06AV.Exigir(permiso.Patente, permiso.Accion);

            // — La operación real —————
            object resultado;
            try { resultado = metodo.Invoke(_interno, argumentos); }
            catch (TargetInvocationException ex)
            { throw ex.InnerException ?? ex; } // ← preserva la excepción de negocio original

            // — Después: auditoría + recálculo del DV —————
            var auditoria = metodo.GetCustomAttribute<AuditaAttribute06AV>();
            if (auditoria != null)
                AuditoriaPcFactory06AV.Registrar(auditoria.Tipo, auditoria.Modulo,
                                                  Describir(metodo, resultado));

            return resultado;
        }

        private static string Describir(MethodInfo m, object resultado)
            => resultado != null ? $"{m.Name}: {resultado}" : m.Name;
    }
}

// 3) La fábrica se simplifica a una línea por servicio
public static ICompraInsumosBLL06AV Compras()
    => InterceptorSeguridad06AV<ICompraInsumosBLL06AV>.Envolver(new CompraInsumosBLL06AV());

```

RECOMENDACIÓN

Para el Trabajo de Diploma conviene **implementar el decorador explícito** en al menos un servicio: es el que se puede diagramar, explicar y defender con el diagrama de clases de GoF al lado. El proxy dinámico usa reflexión y atributos, que son mecanismos de plataforma y no del patrón — vale mencionarlo como optimización, pero no como la implementación principal. Además tiene contras reales: pierde verificación en tiempo de compilación, complica el depurado, y `TargetInvocationException` hay que desenvolverla a mano para no romper el manejo de excepciones que ya existe (está resuelto en el código de arriba).

Compensación manual duplicada → Command

Reserva y consumo de stock, ocupación de líneas de ensamblaje.

5.1 Diagnóstico

PC Factory tiene un modelo de stock en dos niveles que es correcto y bien pensado: `Stock` (unidades físicas), `StockReservado` (comprometidas por ventas cuya orden todavía no cerró) y `StockLibre` como diferencia. La venta *reserva*, el cierre de la orden de producción *consume*. Ese diseño exige que las operaciones sobre stock sean reversibles, y el código efectivamente las revierte.

El problema es **cómo**. La reversión está implementada tres veces, a mano, con la misma estructura: una lista que acumula lo ya hecho, un `try/catch` que la recorre al revés, y un método privado de compensación distinto en cada archivo.

ANTES · Tres copias de la misma compensación manual

```
// — BLL/VentasBLL06AV.cs — compensación manual, copia 1 —
var reservados = new List<KeyValuePair<string, int>>();
try
{
    foreach (var r in requeridos)
    {
        _componentesMpp.ReservarStock(r.Key, r.Value);
        reservados.Add(r); // ← el "diario" de lo hecho, a mano
    }
}
catch (Exception ex)
{
    LiberarReservas(reservados); // ← deshacer, a mano
    throw new AccesoDatosException06AV("No se pudieron reservar los componentes; la venta no se registró.", ex);
}

var venta = new Venta06AV { ... };
try { _mpp.AgregarVenta(venta); }
catch (Exception ex)
{
    LiberarReservas(reservados); // ← deshacer, OTRA VEZ a mano
    throw new AccesoDatosException06AV("No se pudo registrar la venta (se liberó la reserva).", ex);
}

// — BLL/OrdenProduccionBLL06AV.cs — copia 2, misma estructura, otro verbo —
var consumidos = new List<KeyValuePair<string, int>>();
try
{
    foreach (var r in requeridos) { _componentesMpp.ConsumirReserva(r.Key, r.Value); consumidos.Add(r); }
}
catch (Exception ex)
{
    RevertirConsumo(consumidos);
    throw new AccesoDatosException06AV("No se pudo descontar el stock; la orden no se cerró.", ex);
}

string numeroSerie = GenerarNumeroSerie(orden);
```

```
try { _mpp.Cerrar(numeroOrden, numeroSerie); }
catch (Exception ex)
{
    RevertirConsumo(consumidos);           // ← copia 3
    throw new AccesoDatosException06AV("No se pudo cerrar la orden (se restituyó el stock).", ex);
}

LiberarLinea(orden); // ← y este paso NO participa de la compensación: si algo
                    // fallara después, la línea ya quedó liberada. Es un
                    // agujero real en la consistencia del circuito.
```

Hay además un dato que cambia la naturaleza del problema: **no se usa `SqlTransaction` en ningún punto del repositorio**. Cada llamada al DAL abre su propia conexión y la cierra. Es decir que la compensación en memoria no es una red de seguridad adicional: **es el único mecanismo de consistencia del sistema**. Eso eleva considerablemente lo que está en juego en que esté bien hecha.

UN AGUJERO CONCRETO QUE EL PATRÓN CIERRA

En `RegistrarControlCalidad()`, la llamada a `LiberarLinea(orden)` ocurre *después* del último `try/catch` y no participa de la compensación. Su cuerpo, además, es un `try { ... } catch { }` vacío. Si el cierre de la orden fallara en un punto posterior, la línea de ensamblaje quedaría marcada como disponible mientras la orden sigue abierta — y como el sistema no deja planificar sobre líneas ocupadas, el efecto acumulado de estos desfases es el tipo de bug que aparece semanas después y es imposible de reproducir.

5.2 Fundamento teórico

COMMAND — GOF, PATRÓN DE COMPORTAMIENTO

Intención (GoF, pág. 233): «Encapsular una petición como un objeto, permitiendo parametrizar clientes con distintas peticiones, encolar o registrar peticiones, y soportar operaciones deshacibles.»

Aplicabilidad, punto 4: «soportar deshacer. La operación *Execute* del Command puede almacenar el estado necesario para revertir sus efectos. La interfaz debe agregar una operación *Unexecute*. Los comandos ejecutados se guardan en una lista histórica. Recorrerla hacia atrás llamando *Unexecute* deshace un nivel.»

Consecuencias: ① desacopla quién invoca de quién sabe hacer; ② los comandos son objetos de primera clase, manipulables como cualquier otro; ③ se pueden componer comandos en macro-comandos; ④ agregar un comando nuevo no requiere cambiar clases existentes (OCP).

RELACIÓN CON EL PATRÓN SAGA / COMPENSACIÓN

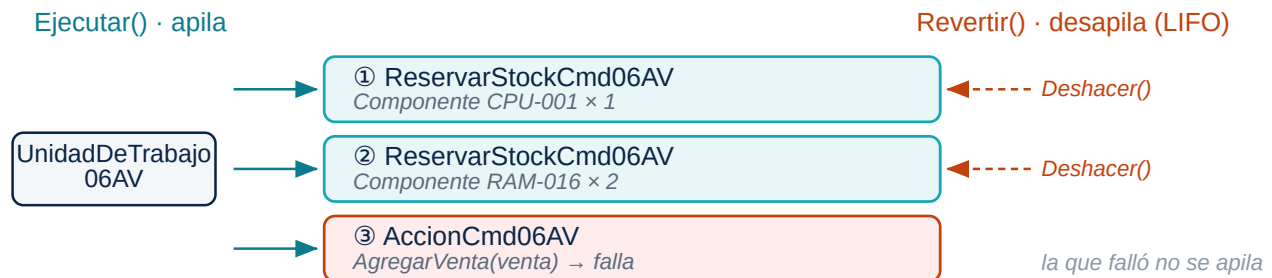
Vale nombrarlo con precisión: lo que hoy hace el código a mano, y lo que el patrón formaliza, es una **transacción compensatoria** —el mecanismo descrito por Garcia-Molina y Salem (1987) para secuencias de operaciones que no pueden compartir una transacción ACID. La diferencia entre "compensación ad-hoc" y "compensación formalizada con Command" es la misma que entre recordar deshacer y que el lenguaje lo garantice.

5.3 Diseño propuesto

El aporte de diseño está en dos puntos:

1. **Hacer y deshacer viven en la misma clase**, a tres líneas de distancia. Hoy `ReservarStock` está en `ComponentesMPP06AV` y su inversa en un método privado de `VentasBLL06AV`, a 150 líneas: es estructuralmente posible modificar una y olvidarse de la otra.
2. **El rollback es automático vía `IDisposable`**. La unidad de trabajo revierte en `Dispose()` si no fue confirmada. Dentro de un bloque `using`, cualquier salida por excepción compensa sola.

NO SE PUEDE OLVIDAR EL CATCH



Sin `Confirmar()`, el `Dispose()` del `using` revierte solo: el rollback no depende de acordarse.
Una compensación que falla no se silencia — queda como error crítico en la bitácora.

Fig. 3 — Historia de comandos y compensación. Al fallar el paso ③ se desapila hacia atrás invocando `Deshacer()` sobre ② y luego sobre ①. El orden inverso es obligatorio: un paso puede depender del anterior.

5.4 Implementación

NUEVO · BLL/Comandos/ – contrato y comandos concretos

```
// =====  
// BLL/Comandos/IComandoCompensable06AV.cs          (NUEVO)  
// El Command de GoF con la extensión "Undo" que el propio libro describe  
// (pág. 237, sección Implementation, punto 3: "Supporting undo and redo").  
// =====  
namespace BLL.Comandos  
{  
    public interface IComandoCompensable06AV  
    {  
        /// <summary>Texto para la bitácora y para el mensaje de error.</summary>  
        string Descripcion { get; }  
  
        void Ejecutar();  
  
        /// <summary>Compensación. Debe ser idempotente y no lanzar: se invoca  
        /// cuando YA hay un error en curso y no puede tapar la causa original.</summary>  
        void Deshacer();  
    }  
}  
  
// =====  
// BLL/Comandos/ComandosStock06AV.cs                (NUEVO)  
// Cada operación reversible del dominio, con su inversa al lado.  
// El acierto es que hacer y deshacer ahora viven en el MISMO archivo, a tres  
// líneas de distancia: es imposible modificar una y olvidarse de la otra.  
// =====
```

```

namespace BLL.Comandos
{
    public sealed class ReservarStockCmd06AV : IComandoCompensable06AV
    {
        private readonly ComponentesMPP06AV _mpp;
        private readonly string _codigo;
        private readonly int _cantidad;

        public ReservarStockCmd06AV(ComponentesMPP06AV mpp, string codigo, int cantidad)
        { _mpp = mpp; _codigo = codigo; _cantidad = cantidad; }

        public string Descripcion => $"reservar {_cantidad} u. de '{_codigo}'";

        public void Ejecutar() => _mpp.ReservarStock(_codigo, _cantidad);
        public void Deshacer() => _mpp.LiberarReserva(_codigo, _cantidad);
    }

    public sealed class ConsumirReservaCmd06AV : IComandoCompensable06AV
    {
        private readonly ComponentesMPP06AV _mpp;
        private readonly string _codigo;
        private readonly int _cantidad;

        public ConsumirReservaCmd06AV(ComponentesMPP06AV mpp, string codigo, int cantidad)
        { _mpp = mpp; _codigo = codigo; _cantidad = cantidad; }

        public string Descripcion => $"consumir {_cantidad} u. reservadas de '{_codigo}'";

        public void Ejecutar() => _mpp.ConsumirReserva(_codigo, _cantidad);

        // Consumir = bajar Stock y bajar StockReservado. Deshacerlo es devolver
        // ambos: el componente vuelve al depósito Y vuelve a estar comprometido.
        public void Deshacer()
        {
            _mpp.SumarStock(_codigo, _cantidad);
            _mpp.ReservarStock(_codigo, _cantidad);
        }
    }

    public sealed class OcuparLineaCmd06AV : IComandoCompensable06AV
    {
        private readonly LineasEnsamblajeMPP06AV _mpp;
        private readonly LineaEnsamblaje06AV _linea;

        public OcuparLineaCmd06AV(LineasEnsamblajeMPP06AV mpp, LineaEnsamblaje06AV linea)
        { _mpp = mpp; _linea = linea; }

        public string Descripcion => $"ocupar la línea #{_linea.Id}";

        public void Ejecutar() { _linea.Disponible = false; _mpp.Modificar(_linea); }
        public void Deshacer() { _linea.Disponible = true; _mpp.Modificar(_linea); }
    }

    /// <summary>Comando genérico para envolver una operación de persistencia que
    /// no necesita inversa (la última de la secuencia, típicamente).</summary>
    public sealed class AccionCmd06AV : IComandoCompensable06AV
    {
        private readonly Action _hacer, _deshacer;
        public AccionCmd06AV(string descripcion, Action hacer, Action deshacer = null)
        { Descripcion = descripcion; _hacer = hacer; _deshacer = deshacer; }

        public string Descripcion { get; }
        public void Ejecutar() => _hacer();
        public void Deshacer() => _deshacer?.Invoke();
    }
}

```

```
}  
}
```

NUEVO · BLL/Comandos/UnidadDeTrabajo06AV.cs – el Invoker

```
// =====  
// BLL/Comandos/UnidadDeTrabajo06AV.cs (NUEVO)  
// El "Invoker" de GoF: mantiene la historia de comandos ejecutados y sabe  
// deshacerla en orden inverso. Implementa IDisposable para que el rollback  
// sea AUTOMÁTICO: si el bloque using sale por excepción sin haber confirmado,  
// se compensa solo. Es imposible olvidarse el catch.  
// =====  
namespace BLL.Comandos  
{  
    public sealed class UnidadDeTrabajo06AV : IDisposable  
    {  
        private readonly Stack<IComandoCompensable06AV> _hechos =  
            new Stack<IComandoCompensable06AV>();  
        private bool _confirmada;  
  
        /// <summary>Ejecuta y registra. Si el comando falla, revierte TODO lo anterior  
        /// y relanza con el detalle de qué paso rompió.</summary>  
        public void Ejecutar(IComandoCompensable06AV comando)  
        {  
            if (_confirmada)  
                throw new InvalidOperationException("La unidad de trabajo ya fue confirmada.");  
  
            try  
            {  
                comando.Ejecutar();  
                _hechos.Push(comando);  
            }  
            catch (Exception ex)  
            {  
                Revertir();  
                throw new AccesoDatosException06AV(  
                    $"Falló la operación '{comando.Descripcion}'. Se revirtieron los pasos previos. " +  
                    $"Detalle: {ex.Message}", ex);  
            }  
        }  
  
        /// <summary>Todo salió bien: la historia se descarta y no habrá rollback.</summary>  
        public void Confirmar() { _confirmada = true; _hechos.Clear(); }  
  
        /// <summary>Deshace en orden INVERSO (LIFO). Cada Deshacer se aísla: si uno  
        /// falla no impide compensar los demás ni oculta la excepción original.</summary>  
        public void Revertir()  
        {  
            while (_hechos.Count > 0)  
            {  
                var comando = _hechos.Pop();  
                try { comando.Deshacer(); }  
                catch (Exception ex)  
                {  
                    // Compensación fallida = inconsistencia real. No se traga en  
                    // silencio como hoy: queda en la bitácora para poder auditarla.  
                    try  
                    {  
                        new BitacoraBLL06AV().ErrorCritico(  
                            $"No se pudo compensar '{comando.Descripcion}': {ex.Message}",  
                            ModuloBitacora.Sistema);  
                    }  
                }  
            }  
        }  
    }  
}
```

```

        catch { }
    }
}

public void Dispose() { if (!_confirmada) Revertir(); }
}
}

```

DESPUÉS · Los dos casos de uso críticos

```

// =====
// BLL/VentasBLL06AV.cs      (DESPUÉS) – CU01 Registrar venta
// Tres try/catch anidados y dos llamadas a LiberarReservas se reducen a un
// bloque using. La intención queda a la vista: "estos N pasos van juntos".
// =====
public Venta06AV RegistrarVenta(Cliente06AV cliente, Computadora06AV computadora, DateTime
fechaEntrega)
{
    // ... validaciones previas (ver Chain of Responsibility en el informe anterior) ...
    var requeridos = Requerimientos(computadora);

    var venta = new Venta06AV
    {
        Cliente = cliente,
        Computadora = computadora,
        FechaEntregaEstimada = fechaEntrega,
        UsuarioRegistro = UsuarioActual()
    };

    using (var trabajo = new UnidadDeTrabajo06AV())
    {
        foreach (var r in requeridos)
            trabajo.Ejecutar(new ReservarStockCmd06AV(_componentesMpp, r.Key, r.Value));

        trabajo.Ejecutar(new AccionCmd06AV(
            $"registrar la venta del cliente {cliente.Dni}",
            hacer: () => _mpp.AgregarVenta(venta),
            deshacer: () => _mpp.AnularVenta(venta.NumeroVenta));

        trabajo.Confirmar();    // ← sin esta línea, el Dispose revierte todo
    }

    return venta;
}

// =====
// BLL/OrdenProduccionBLL06AV.cs      (DESPUÉS) – CU06 Cerrar orden
// Acá el patrón además CORRIGE UN BUG: la liberación de la línea entra en la
// unidad de trabajo, así que si el cierre falla la línea no queda suelta.
// =====
public string RegistrarControlCalidad(int numeroOrden, ControlCalidad06AV control)
{
    var orden = ObtenerOrden0Excepcion(numeroOrden);
    var destino = Transicion(() => control.Aprobado
        ? orden.EstadoActual.Aprobar()    // ← State ($4)
        : orden.EstadoActual.Rechazar());

    _mpp.RegistrarControlCalidad(numeroOrden, control);

    if (!control.Aprobado)                // // escenario alternativo 3.1
    {

```

```

        CambiarEstado(numeroOrden, destino.Valor);
        return null;
    }

    string numeroSerie = GenerarNumeroSerie(orden);

    using (var trabajo = new UnidadDeTrabajo06AV())
    {
        foreach (var r in VentasBLL06AV.Requerimientos(orden.Computadora))
            trabajo.Ejecutar(new ConsumirReservaCmd06AV(_componentesMpp, r.Key, r.Value));

        trabajo.Ejecutar(new AccionCmd06AV(
            $"cerrar la orden #{numeroOrden}",
            hacer: () => _mpp.Cerrar(numeroOrden, numeroSerie),
            deshacer: () => _mpp.Reabrir(numeroOrden)));

        // ANTES quedaba fuera de la compensación. Ahora si algo falla, vuelve a ocuparse.
        if (orden.LineaEnsamblaje != null)
            trabajo.Ejecutar(new LiberarLineaCmd06AV(_lineasMpp, orden.LineaEnsamblaje));

        trabajo.Confirmar();
    }

    orden.TransicionarA(destino);
    return numeroSerie;
}

```

5.5 Decisiones de diseño a justificar

POR QUÉ `DESHACER()` NO PUEDE LANZAR

La compensación se ejecuta cuando ya hay una excepción en curso. Si un `Deshacer()` lanzara, taparía la causa original y el usuario vería un error que no tiene nada que ver con lo que falló. Por eso `Revertir()` aísla cada compensación en su propio `try`. Pero —y esta es la mejora sobre el código actual— **no la silencia**: una compensación fallida es una inconsistencia real de datos, y queda registrada como error crítico en la bitácora. Hoy los `catch { }` de `LiberarReservas` y `LiberarLinea` la pierden por completo.

ORDEN LIFO

La historia es una `Stack`, no una `List`. Es obligatorio: si el paso 3 depende del 2 y el 2 del 1, deshacerlos en orden directo puede fallar o dejar estado inconsistente. GoF lo especifica al describir la lista histórica, y es la misma razón por la que el *rollback* de un motor transaccional recorre el log hacia atrás.

CONVIVENCIA CON TRANSACCIONES REALES

Una vez resuelto el Dolor IV (capítulo 6), el esqueleto del DAL queda centralizado y se puede introducir `SqlTransaction` en un solo lugar. Cuando eso ocurra, **Command no se vuelve redundante: cambia de rol**. La transacción cubre lo que pasa dentro de la base; Command cubre lo que la transacción no puede deshacer —un archivo PDF de comprobante ya generado, un correo enviado, o simplemente una secuencia que abarca varias conexiones. Son complementarios, y así conviene presentarlo.

EXTENSIÓN NATURAL: DESHACER DE USUARIO

Con la historia de comandos ya construida, `VolverAtras()` de la orden de producción —que hoy es un `switch` que reconstruye el paso anterior a mano— podría implementarse como un *undo* real sobre la pila. Es

el caso de uso que GoF describe textualmente. Excede el alcance de este documento pero vale mencionarlo como trabajo futuro en la defensa.

Duplicación en la capa DAL → Template Method

Acceso a datos por procedimientos almacenados.

6.1 Diagnóstico

El bloque de acceso a datos —abrir conexión, armar el `SqlCommand`, cargar parámetros, ejecutar, cerrar— está copiado literalmente en **11 de las 15** clases de la capa DAL. Son unas 30 líneas por archivo, alrededor de 330 líneas duplicadas.

Pero la duplicación no es el peor problema. El peor problema es que **las 11 copias comparten el mismo defecto**: `conn.Close()` se invoca como última sentencia del método, fuera de un `finally` y sin `using`. Si `Fill()` o `ExecuteNonQuery()` lanzan —cualquier error de SQL, un timeout, una violación de restricción— la conexión nunca se cierra ni se libera.

ANTES · El bloque duplicado 11 veces, con el defecto incluido

```
// — DAL/ClientesDAL06AV.cs (y 10 archivos más, IDÉNTICO) —
#region Helpers

private DataTable EjecutarSP(string nombreSP, Dictionary<string, object> parametros)
{
    SqlConnection conn = Conexion.Instancia.ObtenerConexion();
    SqlCommand cmd = new SqlCommand(nombreSP, conn) { CommandType = CommandType.StoredProcedure };
    if (parametros != null)
        foreach (var p in parametros) cmd.Parameters.AddWithValue(p.Key, p.Value);
    DataTable tabla = new DataTable();
    conn.Open();
    new SqlDataAdapter(cmd).Fill(tabla);
    conn.Close();           // ← si Fill lanza, esto NO se ejecuta: conexión filtrada
    return tabla;
}

private void EjecutarSPNonQuery(string nombreSP, Dictionary<string, object> parametros)
{
    SqlConnection conn = Conexion.Instancia.ObtenerConexion();
    SqlCommand cmd = new SqlCommand(nombreSP, conn) { CommandType = CommandType.StoredProcedure };
    if (parametros != null)
        foreach (var p in parametros) cmd.Parameters.AddWithValue(p.Key, p.Value);
    conn.Open();
    cmd.ExecuteNonQuery();
    conn.Close();           // ← mismo problema
}

#endregion

// Presente en: ClientesDAL, ComponentesDAL, ComprasDAL, FamiliaDAL,
// LineasEnsamblajeDAL, ModelosEstandarDAL, PatentesDAL, ProduccionDAL,
// ProveedoresDAL, RolesDAL, VentasDAL → 11 copias del mismo bloque.
```

POR QUÉ TODAVÍA NO EXPLOTÓ

Porque `Conexion.ObtenerConexion()` devuelve una `SqlConnection` nueva en cada llamada y el *connection pooling* de ADO.NET termina recuperando los objetos cuando el recolector de basura los finaliza. Es decir: el sistema se sostiene sobre un comportamiento del entorno, no sobre una garantía del código. Bajo carga —varias estaciones de trabajo operando a la vez, que es precisamente el escenario de PC Factory— el pool se agota antes de que el GC actúe, y el síntoma es un *timeout* al obtener conexión, que además aparece lejos del código que lo causó.

6.2 Fundamento teórico

TEMPLATE METHOD — GOF, PATRÓN DE COMPORTAMIENTO

Intención (GoF, pág. 325): «Definir el esqueleto de un algoritmo en una operación, delegando algunos pasos a las subclasses. Permite que las subclasses redefinan ciertos pasos sin cambiar la estructura del algoritmo.»

Aplicabilidad, punto 2: «cuando el comportamiento común entre subclasses debe factorizarse y localizarse en una clase común para evitar la duplicación de código. Es un buen ejemplo de *refactorizar para generalizar*: se identifican las diferencias en el código existente y se separan en operaciones nuevas.» Esa frase describe este caso con exactitud.

Consecuencias: el libro señala que Template Method «conduce a una estructura de control invertida, a veces llamada *el principio de Hollywood*: no nos llames, nosotros te llamamos» — la clase base llama a los *hooks* de la subclase, nunca al revés.

PRECISIÓN CONCEPTUAL: ¿ES TEMPLATE METHOD O ES SÓLO EXTRAER UNA CLASE BASE?

La distinción importa para la defensa. Mover los dos métodos duplicados a una clase base *no es un patrón*: es la refactorización *Pull Up Method* de Fowler, y se llama DRY. Se convierte en Template Method cuando la clase base define un **algoritmo invariante con puntos de variación explícitos**.

Por eso el diseño propuesto no se limita a subir el código: define el método `Ejecutar<T>` como esqueleto `sealed` —abrir, parametrizar, ejecutar, cerrar, instrumentar— con tres *hooks* virtuales (`TimeoutSegundos`, `ConfigurarComando`, `AlFinalizar`). El paso que varía de verdad —qué se hace con el comando ya preparado— se inyecta como delegado, lo que técnicamente es un **Strategy embebido dentro del Template Method**. Ambas lecturas son correctas y conviene nombrar las dos.

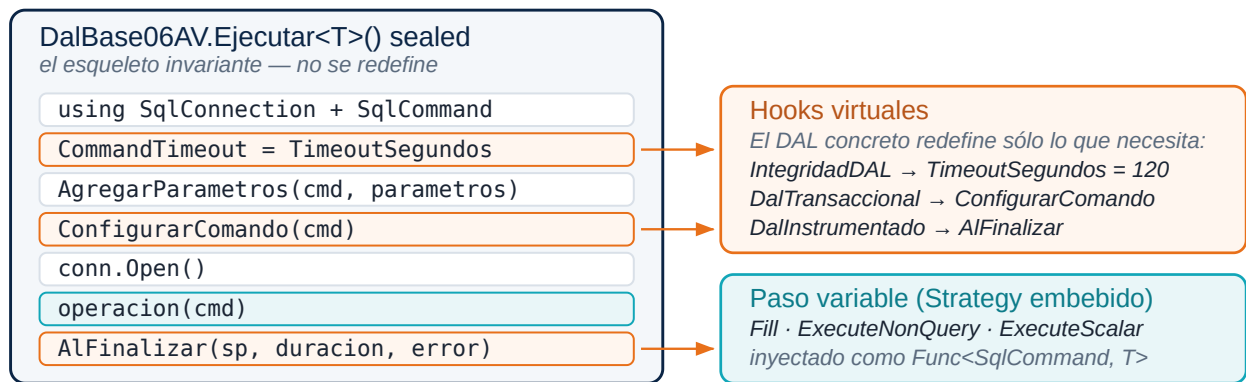


Fig. 4 — Esqueleto invariante y puntos de variación. Lo que convierte esto en Template Method y no en una simple clase base es la existencia de los *hooks*: el algoritmo está fijo y cerrado, y la subclase sólo puede intervenir en los puntos previstos. Es el *principio de Hollywood* que menciona GoF.

6.3 Implementación

NUEVO · DAL/DalBase06AV.cs

```

// =====
// DAL/DalBase06AV.cs                                (NUEVO)
// Template Method: el ESQUELETO del acceso a datos es invariante –
//   abrir conexión → armar comando → parametrizar → [hook] → ejecutar → cerrar
// y el único paso que varía (cómo se consume el comando) se inyecta como
// delegado. Los hooks virtuales permiten a un DAL concreto afinar timeout,
// transacción o logging sin reescribir el esqueleto.
// =====
namespace DAL
{
    public abstract class DalBase06AV
    {
        /// <summary>Hook: segundos de timeout. Un DAL con SP pesados lo sobrescribe.</summary>
        protected virtual int TimeoutSegundos => 30;

        /// <summary>Hook: última oportunidad de tocar el SqlCommand antes de ejecutar
        /// (parámetros OUTPUT, transacción, tipos explícitos...). Vacío por defecto.</summary>
        protected virtual void ConfigurarComando(SqlCommand cmd) { }

        /// <summary>Hook: se llama siempre, haya error o no. Punto natural para
        /// instrumentar tiempos o dejar traza en la bitácora.</summary>
        protected virtual void AlFinalizar(string nombreSP, TimeSpan duracion, Exception error) { }

        // — EL TEMPLATE METHOD —
        // sealed: el esqueleto no se redefine. Lo que varía son los hooks y el
        // delegado 'operacion'. Es exactamente la restricción que GoF recomienda
        // (pág. 327): "the template method itself should not be overridden".
        protected T Ejecutar<T>(string nombreSP,
                                IDictionary<string, object> parametros,
                                Func<SqlCommand, T> operacion)
        {
            var reloj = Stopwatch.StartNew();
            Exception fallo = null;

            // using: la conexión y el comando se liberan SIEMPRE, incluso con excepción.
            // Esto corrige el filtrado de conexiones del código actual.
            using (SqlConnection conn = Conexion.Instancia.ObtenerConexion())

```

```

using (SqlCommand cmd = new SqlCommand(nombreSP, conn))
{
    try
    {
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.CommandTimeout = TimeoutSegundos;
        AgregarParametros(cmd, parametros);
        ConfigurarComando(cmd); // ← hook

        conn.Open();
        return operacion(cmd); // ← paso variable
    }
    catch (SqlException ex)
    {
        fallo = ex;
        // Traducción única: la capa superior recibe un error de dominio,
        // no un SqlException crudo. Antes cada BLL lo envolvía a mano.
        throw new AccesoDatosException06AV(
            $"Error de base de datos al ejecutar '{nombreSP}'.", ex);
    }
    catch (Exception ex) { fallo = ex; throw; }
    finally { reloj.Stop(); AlFinalizar(nombreSP, reloj.Elapsed, fallo); }
}

// — Operaciones concretas, expresadas sobre el template —
protected DataTable EjecutarSP(string nombreSP, IDictionary<string, object> parametros = null)
    => Ejecutar(nombreSP, parametros, cmd =>
    {
        var tabla = new DataTable();
        new SqlDataAdapter(cmd).Fill(tabla);
        return tabla;
    });

protected int EjecutarSPNonQuery(string nombreSP, IDictionary<string, object> parametros =
null)
    => Ejecutar(nombreSP, parametros, cmd => cmd.ExecuteNonQuery());

protected object EjecutarSPEscalar(string nombreSP, IDictionary<string, object> parametros =
null)
    => Ejecutar(nombreSP, parametros, cmd => cmd.ExecuteScalar());

/// <summary>Variante con parámetros OUTPUT (la usa sp_OrdenesCompra_Agregar).</summary>
protected void EjecutarSPConSalida(string nombreSP, IDictionary<string, object> parametros,
    params SqlParameter[] salidas)
    => Ejecutar<object>(nombreSP, parametros, cmd =>
    {
        foreach (var s in salidas) cmd.Parameters.Add(s);
        cmd.ExecuteNonQuery();
        return null;
    });

private static void AgregarParametros(SqlCommand cmd, IDictionary<string, object> parametros)
{
    if (parametros == null) return;
    foreach (var p in parametros)
        cmd.Parameters.AddWithValue(p.Key, p.Value ?? DBNull.Value);
}
}

```

La propiedad clave para la migración: **las firmas de `EjecutarSP` y `EjecutarSPNonQuery` son las mismas que las de los métodos privados actuales**. Por eso el cambio en cada DAL concreto se reduce a agregar la herencia y borrar el bloque duplicado — el cuerpo de la clase no se toca.

DESPUÉS · DAL/ClientesDAL06AV.cs y lo que el patrón habilita

```
// =====
// DAL/ClientesDAL06AV.cs      (DESPUÉS)
// Sólo cambia la declaración de clase y se borra el #region Helpers entero.
// Ni una llamada del cuerpo se modifica: las firmas de EjecutarSP y
// EjecutarSPNonQuery son las mismas. Es una migración de bajísimo riesgo.
// =====
public class ClientesDAL06AV : DalBase06AV           // ← ÚNICO cambio estructural
{
    public DataTable ObtenerTodos()
        => EjecutarSP("sp_Clientes_ObtenerTodos");

    public DataTable ObtenerPorDni(string dni)
        => EjecutarSP("sp_Clientes_ObtenerPorDni",
            new Dictionary<string, object> { { "@Dni", dni } });

    public void Agregar(string dni, string nombre, string apellido, string telefono, string direccion)
        => EjecutarSPNonQuery("sp_Clientes_Agregar", new Dictionary<string, object>
        {
            { "@Dni", dni }, { "@Nombre", nombre }, { "@Apellido", apellido },
            { "@Telefono", telefono }, { "@Direccion", direccion }
        });

    // ← se borran las 30 líneas del #region Helpers
}

// =====
// Lo que HABILITA el patrón: extensiones imposibles antes
// =====

// 1) Un DAL con SP pesados ajusta su timeout sin tocar a nadie más
public class IntegridadDAL06AV : DalBase06AV
{
    protected override int TimeoutSegundos => 120;    // el recálculo de DV recorre todo
}

// 2) Instrumentación transversal del acceso a datos, en UN punto
public abstract class DalInstrumentado06AV : DalBase06AV
{
    protected override void AlFinalizar(string nombreSP, TimeSpan duracion, Exception error)
    {
        if (error == null && duracion.TotalMilliseconds > 500)
            new BitacoraBLL06AV().Error($"SP lento: {nombreSP} ({duracion.TotalMilliseconds:0} ms)",
                ModuloBitacora.Sistema);
    }
}

// 3) EL PREMIO GORDO: transacciones reales. Con el esqueleto centralizado,
//     agregar soporte de SqlTransaction es tocar UN archivo — y entonces el
//     patrón Command ($6) pasa a ser el complemento para lo que la transacción
//     NO cubre (efectos fuera de la base), en vez de ser el único mecanismo.
public abstract class DalTransaccional06AV : DalBase06AV
{
    [ThreadStatic] private static SqlTransaction _transaccionActual;

    protected override void ConfigurarComando(SqlCommand cmd)
    {

```

```

if (_transaccionActual != null)
{
    cmd.Connection = _transaccionActual.Connection;
    cmd.Transaction = _transaccionActual;
}
}
}

```

6.4 Lo que se gana, más allá de borrar 330 líneas

Capacidad	Antes	Después
Liberación de recursos	La conexión se filtra si el SP lanza. 11 veces.	<code>using</code> anidado en el esqueleto: siempre se libera. 1 vez.
Traducción de errores	<code>SqlException</code> sube cruda; cada BLL la envuelve a mano en <code>AccesoDatosException06AV</code> .	Traducida en la frontera de la capa, en un solo punto.
Timeout por módulo	Imposible sin editar 11 archivos.	Un <code>override</code> de una línea en el DAL que lo necesite.
Instrumentación	No existe. Nadie sabe qué SP es lento.	Hook <code>AlFinalizar</code> : SP lentos a la bitácora.
Transacciones	0 usos en el repositorio. Habría que tocar los 15 DAL.	Un solo <code>override</code> de <code>ConfigurarComando</code> .
Parámetros nulos	Cada DAL hace <code>(object)x ?? ""</code> a mano, con criterios distintos.	<code>AgregarParametros</code> normaliza a <code>DBNull.Value</code> en un lugar.

6.5 Migración

Es la más segura de las cuatro y por eso conviene hacerla primero: se puede migrar **un DAL por vez**, sin ningún período de convivencia inestable. Mientras un DAL no esté migrado, conserva sus métodos privados y funciona exactamente como antes.

1. Crear `DalBase06AV`. Nada más. El sistema compila y funciona igual.
2. Migrar `ClientesDAL06AV` (el más simple): agregar `: DalBase06AV`, borrar el `#region Helpers`. Probar el ABM de clientes completo.
3. Repetir para los 10 restantes, uno por *commit*.
4. Casos especiales al final: `ComprasDAL06AV` usa parámetros `OUTPUT` (cubierto por `EjecutarSPConSalida`); `BitacoraDAL06AV`, `UsuariosDAL06AV` e `IntegridadDAL06AV` tienen SQL en línea además de SP y necesitan un `EjecutarTexto` adicional en la base.
5. Verificación transversal: después de migrar todo, hacer un recorrido completo de los 12 casos de uso. Como las firmas no cambiaron, cualquier error sería de compilación, no de runtime.

Plan de implementación

Orden recomendado, dependencias y criterio de verificación.

7.1 Orden y justificación

Etapa	Refactor	Por qué en esta posición	Alcance	Riesgo
1	Template Method en DAL	Es el más mecánico y el único sin dependencias. Corrige un defecto real (fuga de conexiones) y deja el terreno preparado para transacciones, que es lo que le da su forma definitiva a la etapa 3.	15 archivos de <code>DAL/</code> + 1 nuevo	Bajo — las firmas no cambian
2	State	Achica y homogeneiza los métodos del BLL. Hacerlo antes que el Decorator significa que los decoradores se escriben una sola vez sobre métodos ya estabilizados.	<code>BE/Estados/</code> + 2 BLL + 3 controles de UI + 3 JSON de idioma	Medio — toca UI y BLL
3	Command	Depende de la etapa 1 para decidir su alcance definitivo: con transacciones disponibles, Command cubre sólo lo extra-transaccional en lugar de ser el único mecanismo.	<code>BLL/Comandos/</code> + 2 métodos de BLL	Medio — toca consistencia de stock
4	Decorator / Proxy	Último porque es el que más superficie toca (interfaz por cada BLL, más el punto de composición) y porque se beneficia de que las etapas 2 y 3 ya hayan estabilizado las firmas.	6 interfaces + 12 decoradores + 1 fábrica + N controles de UI	Medio-alto — cambia cómo la UI obtiene servicios

ADVERTENCIA SOBRE EL DÍGITO VERIFICADOR

Las cuatro etapas tocan código que escribe en la base, y por lo tanto interactúan con el mecanismo de integridad. Antes de empezar conviene ejecutar `IntegridadBLL06AV.RespaldaEnCarpetaPorDefecto()` y anotar la línea base, y después de cada etapa verificar con `Verificar()` que el DV sigue consistente. La etapa 4 es la crítica: si un decorador de auditoría queda mal compuesto, el recálculo del DV deja de dispararse y el síntoma aparece recién en el próximo *login*, como corrupción falsa.

7.2 Verificación sin framework de pruebas

El proyecto no tiene pruebas automatizadas —el propio `AGENTS.md` lo declara. En ese contexto, la verificación tiene que ser un recorrido manual explícito de los doce casos de uso, y conviene dejarlo documentado en el TD como parte del proceso:

- **RFN1 completo:** CU02 registrar cliente → CU01 registrar venta (verificar que `StockReservado` sube) → CU03 registrar seña → CU04 generar orden → CU05 asignar línea → CU06 control de calidad aprobado

(verificar que `Stock` baja y `StockReservado` también) → CU07 entregar.

- **RFN1 alternativos:** venta sin stock suficiente; control de calidad rechazado → En revisión → segundo control aprobado; anulación de venta antes de producir (verificar que la reserva se libera); `VolverAtras` desde cada estado que lo permita.
- **RFN2 completo:** CU08 registrar orden de compra → CU09 seleccionar proveedor → CU10 generar cotización → CU11 aprobar → CU12 recibir. Más el alternativo de recepción parcial y segunda recepción contra la misma orden.
- **Seguridad (etapa 4):** iniciar sesión con un rol sin la patente `AprobarCotizacion` y verificar que la operación se rechaza *en el BLL*, no sólo por ausencia de botón. Este es el caso de prueba que hoy fallaría y que justifica el patrón.
- **Integridad:** `Verificar()` después de cada etapa; *login* limpio sin falsos positivos de corrupción.

7.3 Qué mostrar en la defensa

Para cada patrón, la secuencia que mejor funciona es la misma que usa este documento: **el código actual, el diagrama de clases de GoF, el código nuevo, y el caso de prueba que antes fallaba**. El cuarto elemento es el que distingue un patrón aplicado de un patrón recitado — y los cuatro de este documento lo tienen:

- **State:** agregar un estado al circuito y mostrar que la UI no se toca.
- **Decorator:** invocar una operación de negocio sin la patente correspondiente y mostrar que ahora se rechaza en la capa correcta.
- **Command:** forzar un fallo en el cierre de una orden y mostrar que la línea de ensamblaje no queda huérfana.
- **Template Method:** forzar un error de SQL y mostrar, en el monitor de conexiones de SQL Server, que ya no se filtran.

Bibliografía

FUENTES PRIMARIAS DE LOS PATRONES

- Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. — State (pág. 305), Decorator (175), Proxy (207), Command (233), Template Method (325).
- Larman, C. (2004). *Applying UML and Patterns* (3.ª ed.). Prentice Hall. — Patrones GRASP: Polimorfismo, Indirección, Variaciones Protegidas, Fabricación Pura.

PRINCIPIOS DE DISEÑO

- Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall. — SRP, OCP, DIP.
- Meyer, B. (1988). *Object-Oriented Software Construction*. Prentice Hall. — Formulación original del Principio Abierto/Cerrado.

REFACTORIZACIÓN Y ARQUITECTURA DE APLICACIONES

- Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2.ª ed.). Addison-Wesley. — *Replace Conditional with Polymorphism* (base del capítulo 3), *Pull Up Method* y *Form Template Method* (capítulo 6).
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley. — *Data Mapper* (la separación DAL/MPP del proyecto), *Unit of Work* (base del capítulo 5), *Layer Supertype* (base del capítulo 6).

TRANSACCIONES COMPENSATORIAS

- Garcia-Molina, H.; Salem, K. (1987). *Sagas*. Proceedings of the ACM SIGMOD International Conference on Management of Data. — Fundamento de la compensación en secuencias no atómicas (capítulo 5).

DOCUMENTACIÓN DEL PROYECTO

- Repositorio [IngSoftValdezAlegre](#) — código fuente analizado.
- [AGENTS.md](#) — arquitectura de capas, convenciones de nomenclatura y advertencias sobre el mecanismo de dígito verificador.
- Material complementario de la cátedra — caso didáctico LEVEL WHITE.